L5: Statistical Arbitrage and Pairs Trading

Course Code: COMP7415

Lecturer: Tony Lam

Agenda

- Statistical arbitrage
  - Pair Trading
  - Co-integration
- Correlation vs Cointegration
- Implement a cointegration strategy
- FX Market Introduction
- USD-HKD arbitrage

Introduction to Statistical Arbitrage

- A trading strategy that seeks to exploit statistical mis-pricings of one or more assets based on historical data.
- It is a mean reversion strategy that profits from the convergence of asset prices.
- Commonly involves pairs trading, basket trading, or market-neutral strategies.
- Due to market-neutral, it reduces exposure to overall market risk of a portfolio.

Top Hedge Funds

- **Renaissance Technologies**: Known for using statistical arbitrage strategies in their Medallion Fund.
- **Two Sigma**: Utilizes machine learning and statistical models to implement market-neutral strategies.

Year	Medallion Fund Net Return	$100 invested in the Medallion Fund (net returns)	S&P 500 Return	$100 invested in the S&P 500
1988	9.04%	$109.04	12.40%	$112.40
1989	-3.20%	$105.55	27.25%	$143.03
1990	58.24%	$167.02	-6.56%	$133.65
1991	39.44%	$232.90	26.31%	$168.81
1992	33.60%	$311.15	4.46%	$176.34
1993	39.12%	$432.87	7.06%	$188.79
1994	70.72%	$739.00	-1.13%	$186.65
1995	38.32%	$1,022.19	33.56%	$249.29
1996	31.52%	$1,344.38	20.26%	$299.80
1997	21.20%	$1,629.39	31.01%	$392.77
1998	41.68%	$2,308.52	26.67%	$497.52
1999	24.48%	$2,873.64	19.53%	$594.69
2000	98.48%	$5,703.61	-10.12%	$534.51
2001	33.02%	$7,586.94	-13.05%	$464.75
2002	25.82%	$9,545.88	-23.37%	$356.14
2003	21.90%	$11,636.43	26.38%	$450.09
2004	24.92%	$14,536.23	8.99%	$490.55
2005	29.51%	$18,825.87	3.00%	$505.27
2006	44.30%	$27,165.74	13.62%	$574.09
2007	73.42%	$47,110.82	3.53%	$594.35
2008	83.38%	$86,391.82	-38.49%	$365.59
2009	38.98%	$120,067.36	23.45%	$451.32
2010	29.40%	$155,367.16	12.78%	$508.99
2011	37.02%	$212,884.08	0.00%	$508.99
2012	29.01%	$274,641.76	13.41%	$577.25
2013	46.93%	$403,531.13	29.60%	$748.12
2014	39.20%	$561,715.34	11.39%	$833.33
2015	36.01%	$763,989.03	-0.73%	$827.24
2016	35.62%	$1,036,121.93	9.54%	$906.16
2017	45.02%	$1,502,584.02	19.42%	$1,082.14
2018	39.98%	$2,103,317.11	-6.24%	$1,014.61

Source: https://www.visualcapitalist.com/growth-of-100-invested-in-jim-simons-medallion-fund/#google_vignette

Pair Trading

Introduction to Pair Trading

- A market-neutral trading strategy that involves trading two correlated assets.
- Exploits the relative price movements between the two assets.
- Example: Buying one stock and simultaneously selling another if their price spread deviates from the historical mean.

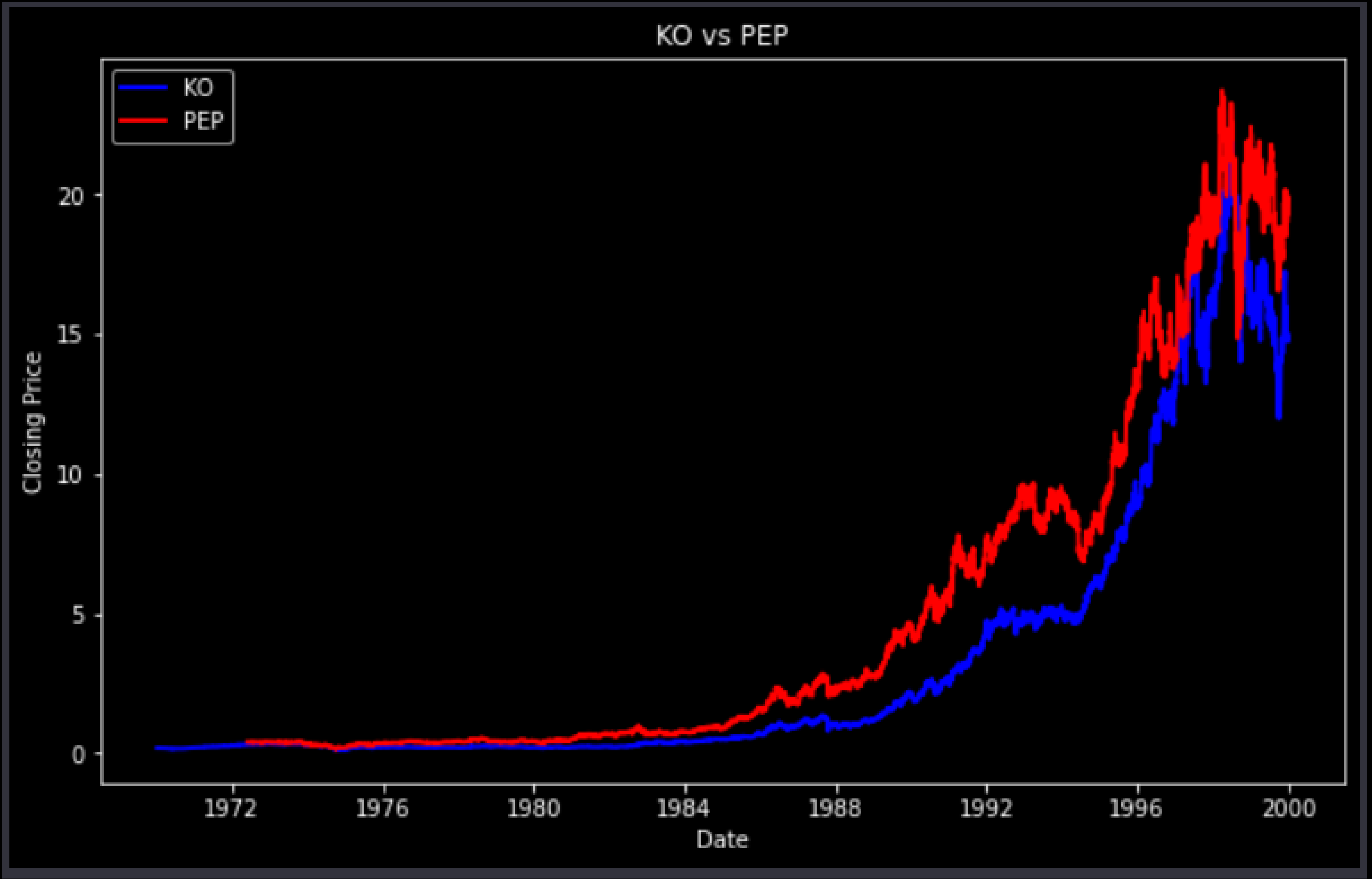
Example: Coca-Cola (KO) and PepsiCo (PEP)

```python
import pandas as pd
import yfinance as yf

# Fetch historical data
data_KO = yf.download('KO', start='1970-01-01', end='1999-12-31')
data_PEP = yf.download('PEP', start='1970-01-01', end='1999-12-31')

# Display the first few rows of the dataset
print(data_KO.head())
print(data_PEP.head())

# Preprocess the data
data_KO['Date'] = data_KO.index
data_KO['Date'] = pd.to_datetime(data_KO['Date'])
data_KO.set_index('Date', inplace=True)

data_PEP['Date'] = data_PEP.index
data_PEP['Date'] = pd.to_datetime(data_PEP['Date'])
data_PEP.set_index('Date', inplace=True)
```

```
Price            Close      High       Low      Open   Volume
Ticker              KO        KO        KO        KO       KO
Date
1970-01-02    0.176391  0.176391  0.175051  0.176391  1104000
1970-01-05    0.173710  0.175855  0.173174  0.175855   556800
1970-01-06    0.175319  0.175855  0.172906  0.173710   796800
1970-01-07    0.176391  0.176927  0.174247  0.175319   700800
1970-01-08    0.176659  0.177999  0.175319  0.176391  1065600

Price            Close      High       Low      Open  Volume
Ticker             PEP       PEP       PEP       PEP     PEP
Date
1972-06-01    0.381617  0.381617  0.377205  0.377205  318600
1972-06-02    0.380514  0.385478  0.380514  0.381617  140400
1972-06-05    0.376090  0.382174  0.368900  0.381621  469800
1972-06-06    0.372771  0.372771  0.371665  0.372771  140400
1972-06-07    0.370559  0.373324  0.369453  0.372771  178200
```

Closing Price: KO vs PEP

```
import matplotlib.pyplot as plt

# Create a new DataFrame with common dates
common_dates = data_KO.index.intersection(data_PEP.index)
data_combined = pd.DataFrame({
    'KO_Close': data_KO['Close'].loc[common_dates]['KO'],
    'PEP_Close': data_PEP['Close'].loc[common_dates]['PEP'] }
)

# Plot the closing prices
plt.style.use('dark_background')
plt.figure(figsize=(10, 6))

plt.plot(data_combined.index, data_combined['KO_Close'],
color='blue', linewidth=2, label='KO')
plt.plot(data_combined.index, data_combined['PEP_Close'],
color='red', linewidth=2, label='PEP')

plt.xlabel('Date')
plt.ylabel('Closing Price')
plt.title('KO vs PEP')
plt.legend()
plt.show()
```



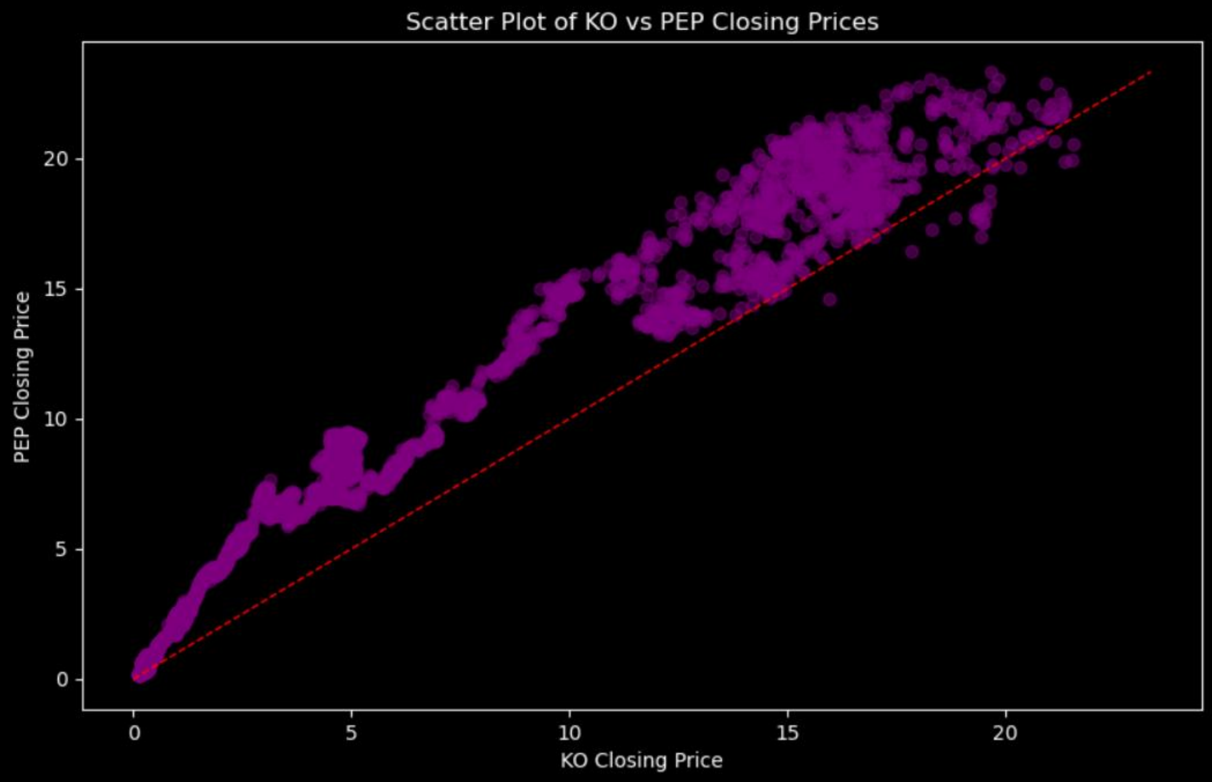
Correlation: KO vs PEP

```python
# Calculate the correlation between KO and PEP
correlation = data_combined['KO_Close'].corr(data_combined['PEP_Close'])
print(f'Correlation between KO and PEP: {correlation:.2f}')
```

```
Correlation between KO and PEP: 0.98
```

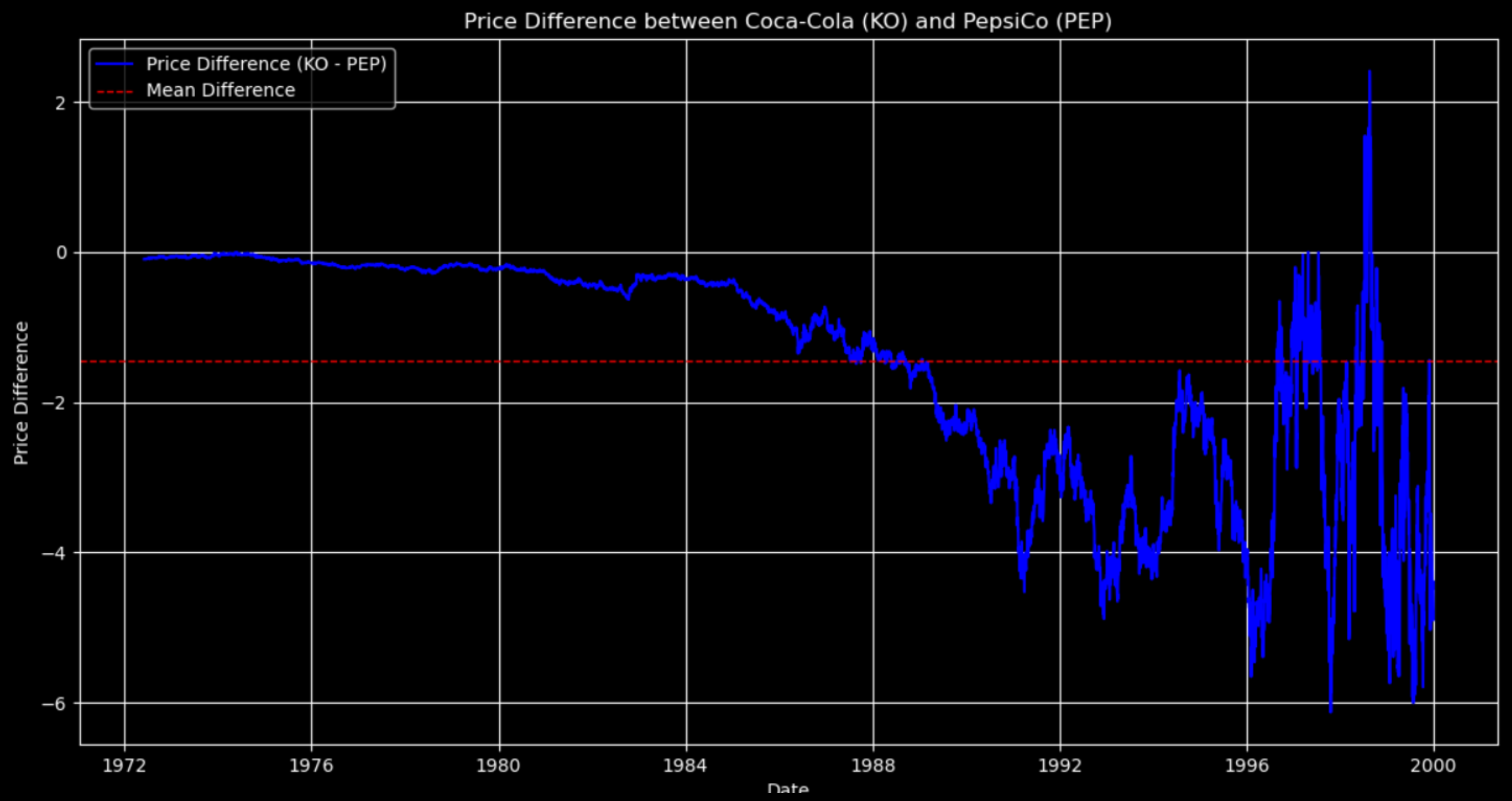
Scatter Plot: KO vs PEP

```
# Scatter plot between KO and PEP closing prices
plt.figure(figsize=(10, 6))
plt.scatter(data_combined['KO_Close'], data_combined['PEP_Close'],
alpha=0.5, color='purple')
plt.title('Scatter Plot of KO vs PEP Closing Prices')
plt.xlabel('KO Closing Price')
plt.ylabel('PEP Closing Price')

# Add a 1:1 line for reference
max_price = max(data_combined['KO_Close'].max(),
data_combined['PEP_Close'].max())
plt.plot([0, max_price], [0, max_price], color='red', linestyle='--',
linewidth=1, label='1:1 Line')
plt.show()
```

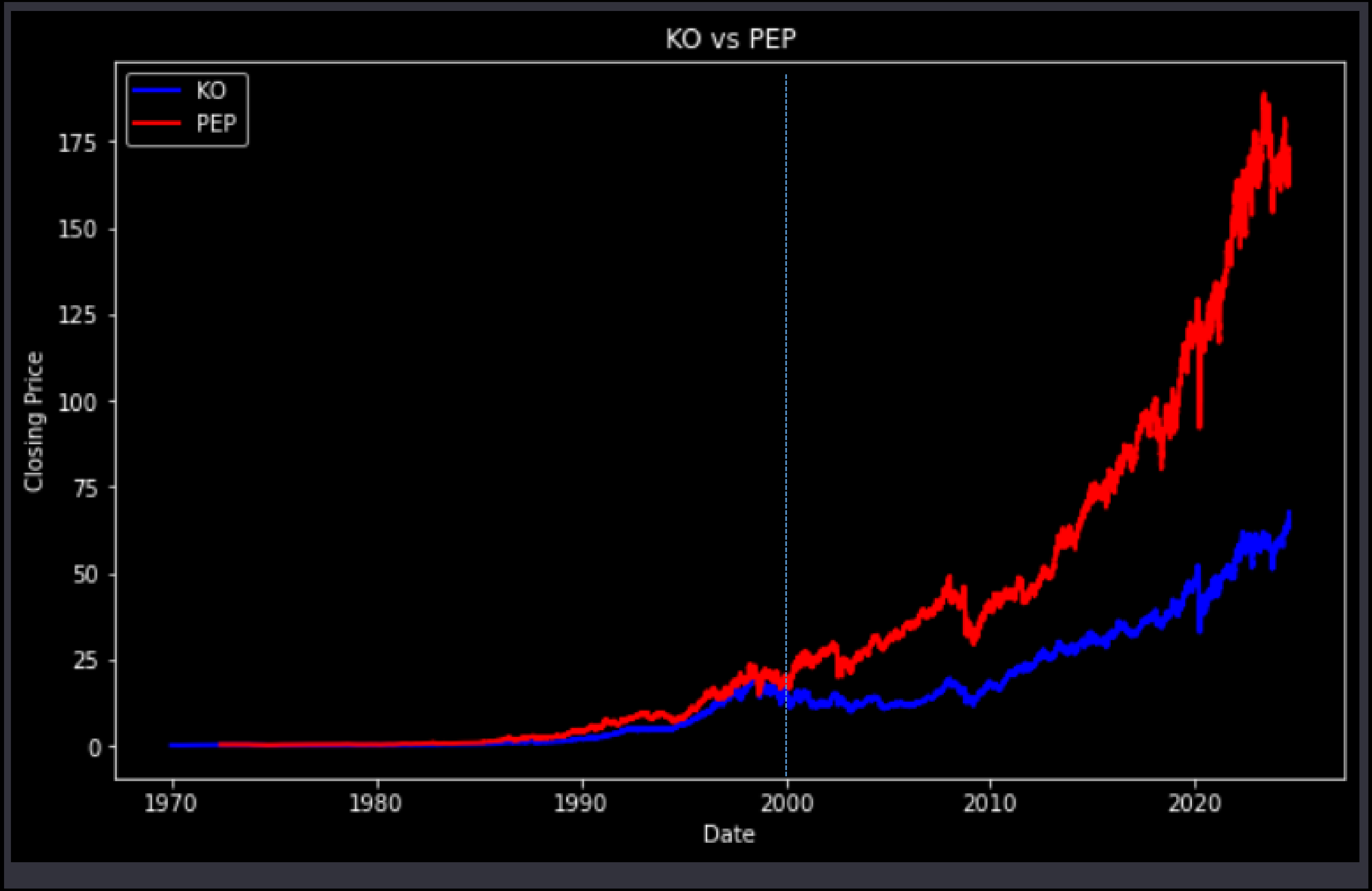


Price Difference: KO vs PEP

```
# Calculate the price difference
price_difference = data_combined['KO_Close'] - 
data_combined['PEP_Close']

# Calculate the mean of the price difference
mean_difference = price_difference.mean()

# Plotting the price difference
plt.figure(figsize=(14, 7))
plt.plot(price_difference.index, price_difference.values, 
label='Price Difference (KO - PEP)', color='blue')
plt.axhline(mean_difference, color='red', linestyle='--', 
linewidth=1, label='Mean Difference') # Mean line
plt.title('Price Difference between Coca-Cola (KO) and 
PepsiCo (PEP)')
plt.xlabel('Date')
plt.ylabel('Price Difference')
plt.legend()
plt.grid()
plt.show()
```



Does this pair still valid nowadays?



Potential reasons

- Fundamental Changes:
  - Company Performance: Changes in the operational performance, financial health, or strategic direction of either KO or PEP can lead to divergence in their stock prices.
  - Market Positioning: If one company captures more market share or shifts its product strategy significantly (e.g., focusing on health-conscious products), it may affect its stock price relative to the other.
- Economic Factors:
  - Economic downturns or inflation can affect consumer spending patterns, impacting KO and PEP differently based on their product offerings and market strategies.
- Investor Behavior:
  - Changes in investor sentiment towards either company (e.g., due to negative news, earnings reports, or analyst downgrades) can lead to price movements that do not correspond with historical patterns.


Brands owned by Coca-Cola
Divided by segment
Other
Coffee & Tea
Alcohol ready-to-drink
Juices, Dairy & Plant-based
Waters & Hydration
Sparkling Soft Drinks
AHA
Barq's
del Valle
BODYARMOR
COSTA COFFEE
GOLD PEAK TEA
GEORGIA
fuze tea
doğadan
FRESCA MIXED
Topo Chico HARD SELTZER
Simply SPIKED
LEMON-DOU
Minute Maid
AdeS
fairlife
innocent
Simply juices and drinks
vitaminwater
smartwater
DASANI
いろはす I LOHAS
Topo Chico
ciel
AQUARIUS
POWERADE
Coke
Sprite
FRESCA
Schweppes
Coca-Cola
Fanta
Appletiser
Source: Coca-Cola Company
Capital at risk.


Everything Owned by PEPSICO
Beverages
Food
International Brands
AQUAFINA
BRISK
bubly
Crush
Dr Pepper
driftwell
EVOLVE
KEVITA
LIFEWTR
GATORADE
Lipton
Naked
MUG ROOT BEER
Ocean Spray
PEPSI
PURE LEAF
Schweppes
sodastream
propel
SoBe
soulboost
STUBBORN
Starry
TAZO
Tropicana
BAKEN-ETS
bare
CAP'N CRUNCH
Cheetos
Cracker Jack
Fritos
Doritos
hilo
Grandma's
MAUI STYLE
Miss Vickie's
Munchos
Lay's
NatuChips
MUNCHIES
Near East
NUT HARVEST
PASTA RONI
POPCORNERS
Pearl Milling Company
RICE A RONI
ROLD GOLD
QUAKER
SIMPLY
RUFFLES
SANTITAS
Smartfood
Stacy's
SUN CHIPS
TOSTITOS
7UP
Alvalle
CHIPSY
Чудо
Duyvis
EVERVESS
Elma Chips
HICKORY STICKS
FRUSTYLE
IZZE
KAS
Kurkure
LOOZA
MARBO
Marias
PASO DE LOS TOROS
PIONEER FOODS
mabel
Sabritas
Sabritones
Sandora
SIERRA MIST
Simba
SMITH'S
Stiksy
Twisties
WALKERS
Я
Note: Some brands listed here are partially owned by PepsiCo, have specific products licensed and distributed by PepsiCo, or are licensed and distributed by PepsiCo in certain markets.

Cointegration

Cointegration

- Most financial time series are not stationary nor mean reverting.
- However, we can create a portfolio of individual price series so that the market value (or price) series of this portfolio is stationary.
- If we can find a stationary linear combination of several non-stationary price series, then these price series are called cointegrated.

Definition and Formula

- Two time series $\{X_t\}, \{Y_t\}$ are cointegrated if there exists a constant β such that $Z_t = X_t - \beta Y_t$ is stationary
- Here β represents the cointegrating coefficient.
- In market practice, β is also called the hedge ratio suggesting the number of shares to long (short) Y for each share short (long) on X

Statistical Tests for Cointegration

- Cointegrated Augmented Dickey Fuller Test (CADF)
- Steps:
  - We first determine the optimal hedge ratio by running a linear regression between two price series
  - Use this hedge ratio to form a portfolio
  - Then run an ADF test on this portfolio price series $\{Z_t = X_t - \beta Y_t\}$
  - If the null hypothesis is rejected, we can conclude that Z_t is stationary, and hence $\{Y, X\}$ are cointegrated

Python Example

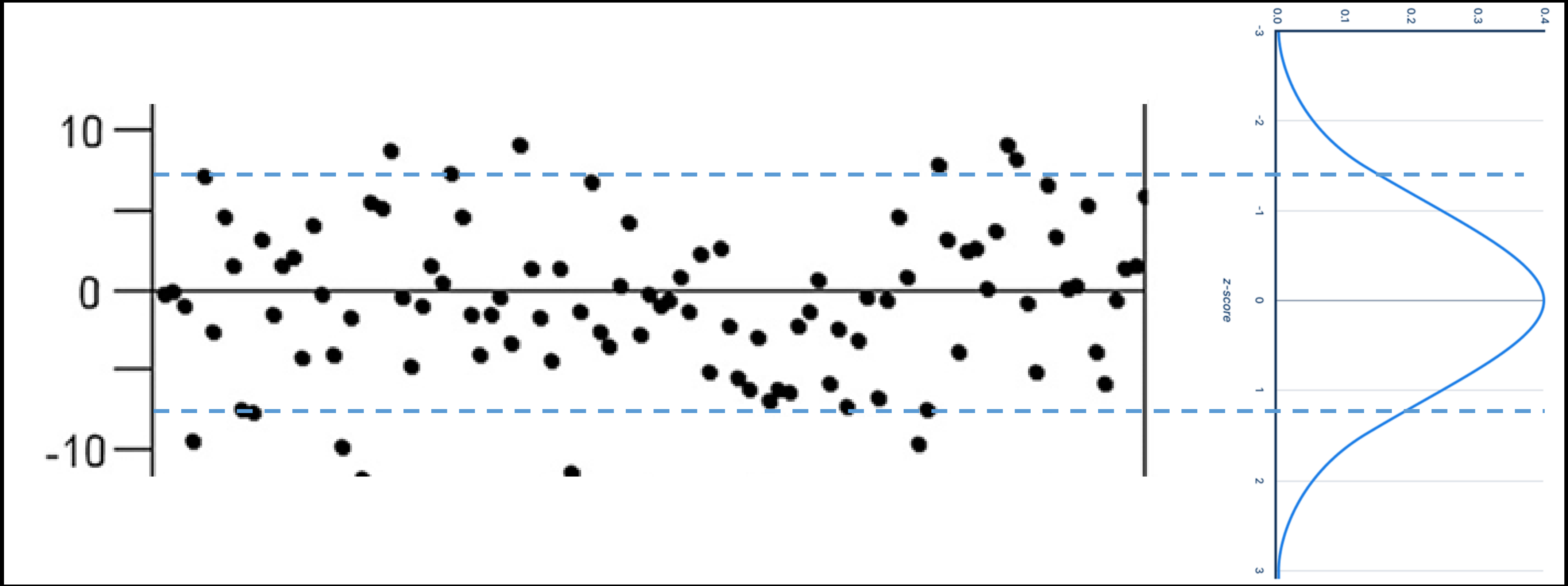
```python
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
import yfinance as yf

start_date = '2020-01-01'
end_date = '2025-08-31'

ko = yf.Ticker("KO").history(start=start_date, end=end_date)
pep = yf.Ticker("PEP").history(start=start_date, end=end_date)

# Price data
ko_close = ko['Close']
pep_close = pep['Close']

# Cointegration test
result = sm.tsa.stattools.coint(ko_close, pep_close)
print('Cointegration test p-value:', result[1])
```

```
Cointegration test p-value: 0.9555567191241595
```

Steps to Implement Pair Trading

1. Identify a pair of correlated assets.
2. Test for co-integration between the assets.
3. Calculate the spread and determine the historical mean and standard deviation.
4. Generate trading signals when the spread deviates from the mean by a certain threshold.
5. Execute trades: Go long on the undervalued asset and short on the overvalued asset.
6. Close positions when the spread reverts to the mean.

Market Entry/Exit based on the Spread's Distance



Key Components

- **Co-integration**: A statistical property indicating a long-term equilibrium relationship between two time series.
- **Spread**: The difference in price between the two assets being traded.
- **Mean Reversion**: The assumption that the spread will revert to its historical average.

Cointegration for multiple variables (more than 2 assets)

Cointegrated Augmented Dickey Fuller Test

- Create a multiple linear regression model

$$Y_t = \beta_0 + \beta_1 X_{1t} + \cdots + \beta_k X_{kt} + \varepsilon_t$$

- Steps:
  - We first determine the optimal hedge ratios by running a multiple linear regression between the price series
  - Use the hedge ratios $\{\beta_i\}$ to form a portfolio
  - Then run an ADF test on this portfolio price series $\{Z_t = Y_t - \beta_0 - \beta_1 X_{1t} - \cdots - \beta_k X_{kt}\}$
  - If null hypothesis is rejected, we can conclude that Z_t is stationary, and hence $\{Y, X_1, \dots, X_k\}$ are cointegrated

Limitation of above idea

- If null hypothesis cannot be rejected, we only know that $\{Y, X_1, \ldots, X_k\}$ cannot form a stationary time series. However, it is possible that a subset (eg. $Y, X_1 \text{ and } X_2$) can be cointegrated. The CADF test doesn't provide further insights about this.
- It only assumes a multiple linear relationship between the stocks, but ignore other useful variables (eg. economic factors, technical indicators, etc).

Vector Error Correction Model (VECM)

- The concept of cointegration for multiple variables can be captured using the Vector Error Correction Model (VECM) framework.
- General Framework:
  1. Vector Autoregressive Model (VAR):

     Consider a vector of multiple non-stationary time series:

$$\boldsymbol{Y_t} = \begin{pmatrix} Y_{1t} \\ \vdots \\ Y_{nt} \end{pmatrix}$$

A VAR model can be expressed as:

$$\boldsymbol{Y_t} = A\boldsymbol{Y_{t-1}} + BX_t + \boldsymbol{\mu_t}$$

Where
- A: Matrix of coefficients.
- X_t: Exogenous variables (if any)
- μ_t: Error term

Vector Error Correction Model (VECM)

- General Framework:
  2. Conintegration

     If the series are cointegrated, there exists a matrix β such that:

$$\beta^T Y_t \sim I(0)$$

     $I(0)$ is a notation referring to a stationary time series without order differencing. This means that a linear combination of the series, given by $\beta^T Y_t$ is stationary

Johansen Test for Cointegration

- It tests for cointegration of more than 2 variables
- Let's generalize the discrete version of the ADF equation.
  - $\overrightarrow{Y_t}$ are vectors representing multiple time series
  - Λ, A_k are matrices

$$\Delta\overrightarrow{Y_t} = \Lambda\overrightarrow{Y_{t-1}} + \vec{M} + A_1\Delta\overrightarrow{Y_{t-1}} + \cdots + A_k\Delta\overrightarrow{Y_{t-k}} + \overrightarrow{\varepsilon_t}$$

- Just in the univariate case, cointegration doesn't exist if Λ=0
- Denote the rank of Λ is *r* and the number of time series is ***n***
  - So testing whether Λ=0 will be equivalent to testing r=0

Johansen Test for Cointegration

- The number of independent portfolios that can be formed by various linear combinations of the cointegrating price series is equal to *r*. The Johansen test will calculate *r* in two different ways, both based on the eigenvector decomposition of Λ.
    1. Trace Statistic
    2. Maximum Eigenvalue Statistic

Johansen Test for Cointegration

- Meaning of r:
  - It represents the number of cointegrating vectors among the time series being analyzed.
    - If r=0: There are no cointegrating relationships; the time series are not stationary in any linear combination.
    - If r>0: There are r cointegrating relationships, indicating that some linear combinations of the time series are stationary, suggesting long-run equilibrium relationships among them.
- As a useful by-product, the eigenvectors found can be used as our hedge ratios for the individual price series to form a stationary portfolio.

Johansen Test for Cointegration

1. Trace Statistic
   - Tests the null hypothesis that the number of cointegrating vectors is less than or equal to r

$$LR_{trace}(r) = -T \sum_{i=r+1}^{k} \log(1 - \widehat{\lambda}_i)$$

where

- T: Sample size
- k: Number of variables in the system
- $\widehat{\lambda_i}$: The estimated eigenvalues from the matrix of Λ

Johansen Test for Cointegration

2. Maximum Eigenvalue Statistic
   - Tests the null hypothesis that there is exactly r cointegrating vectors against the alternative of r+1

$$LR_{max}(r) = -T\ log(1 - \widehat{\lambda_{r+1}}\ )$$

   - It focuses specifically on the $(r+1)^{th}$ eigenvalue

Python Example

- Determine whether the “Magnificent 7”stocks are cointegrated using the daily closing price from 2020 to 2025
  - AAPL
  - AMZN
  - NVDA
  - MSFT
  - TSLA
  - META
  - GOOGL

```python
import numpy as np
import pandas as pd
import yfinance as yf
from statsmodels.tsa.vector_ar.vecm import coint_johansen

# Define the stock symbols
stocks = ['AAPL', 'AMZN', 'NVDA', 'MSFT', 'TSLA', 'META', 'GOOGL']

# Download the stock data
df = yf.download(stocks, start='2020-01-01', end='2025-12-31')['Close']

# Display the first few rows of the data
print("Stock prices data:\n", df.head())

# Perform the Johansen test
result = coint_johansen(df, det_order=0, k_ar_diff=1)

# Check the results
print("\nEigenvalues:\n", result.eig)
print("\nCritical values (trace):\n", result.cvt)
print("\nCritical values (max eigenvalue):\n", result.cvm)

# Determine the rank based on the trace statistic
trace_stat = result.lr1
critical_values = result.cvt[:, 1] # 5% critical values

rank = np.sum(trace_stat > critical_values)

print("\ntrace_stat=",trace_stat)
print("\nNumber of cointegration relationships (rank):", rank)

# Determine the rank based on the maximum eigenvalue statistic
max_eigen_stat = result.lr2
critical_values = result.cvm[:, 1] # 5% critical values

rank = np.sum(max_eigen_stat > critical_values)

print("\nmax_eigen_stat=", max_eigen_stat)
print("\nNumber of cointegration relationships (rank):", rank)
```

Reference: https://www.statsmodels.org/dev/generated/statsmodels.tsa.vector_ar.vecm.coint_johansen.html

```
Stock prices data:
 Ticker           AAPL        AMZN      GOOGL         META        MSFT       NVDA
TSLA
Date
2020-01-02  72.400505  94.900497  67.920815  208.324753  152.505692  5.971076
28.684000
2020-01-03  71.696617  93.748497  67.565483  207.222488  150.606720  5.875504
29.534000
2020-01-06  72.267921  95.143997  69.366386  211.125244  150.995987  5.900144
30.102667
2020-01-07  71.928047  95.343002  69.232391  211.582031  149.619278  5.971575
31.270666
2020-01-08  73.085098  94.598503  69.725174  213.727051  152.002472  5.982776
32.809334

Eigenvalues:
 [0.02747738 0.02070775 0.011672   0.00785587 0.00530318 0.00366997
 0.00042318]

Critical values (trace):
 [[120.3673 125.6185 135.9825]
 [ 91.109   95.7542 104.9637]
 [ 65.8202  69.8189  77.8202]
 [ 44.4929  47.8545  54.6815]
 [ 27.0669  29.7961  35.4628]
 [ 13.4294  15.4943  19.9349]
 [  2.7055   3.8415   6.6349]]

Critical values (max eigenvalue):
 [[43.2947 46.2299 52.3069]
 [37.2786 40.0763 45.8662]
 [31.2379 33.8777 39.3693]
 [25.1236 27.5858 32.7172]
 [18.8928 21.1314 25.865 ]
 [12.2971 14.2639 18.52  ]
 [ 2.7055  3.8415  6.6349]]

trace_stat= [117.13703785  75.20481929  43.71245014  26.0427652   14.1730027
   6.17048494   0.63701792]

Number of cointegration relationships (rank): 0

max_eigen_stat= [41.93221855 31.49236916 17.66968493 11.8697625   8.00251776
 5.53346703  0.63701792]

Number of cointegration relationships (rank): 0
```

(90%, 95%, 99%)
confidence level

Cointegration vs Correlation

Correlation

- Measures the strength and direction of a linear relationship between 2 variables.
- The value is ranging between -1 and 1
  - 1: Perfect positive linear relationship
  - -1: Perfect negative linear relationship
  - 0: No linear relationship
- **Limitation**: Does not imply a long-term equilibrium relationship.

Cointegration

- Indicates a long-term equilibrium relationship between 2 or more time series, even if they are non-stationary individually.
- If two series are cointegrated, they will move together over time, despite short-term deviations.

Key Differences

- **Nature**:
  - Correlation is about the short-term relationship.
  - Cointegration focuses on the long-term relationship.
- **Application**:
  - Use correlation for diversification.
  - Use cointegration for identifying tradable pairs.

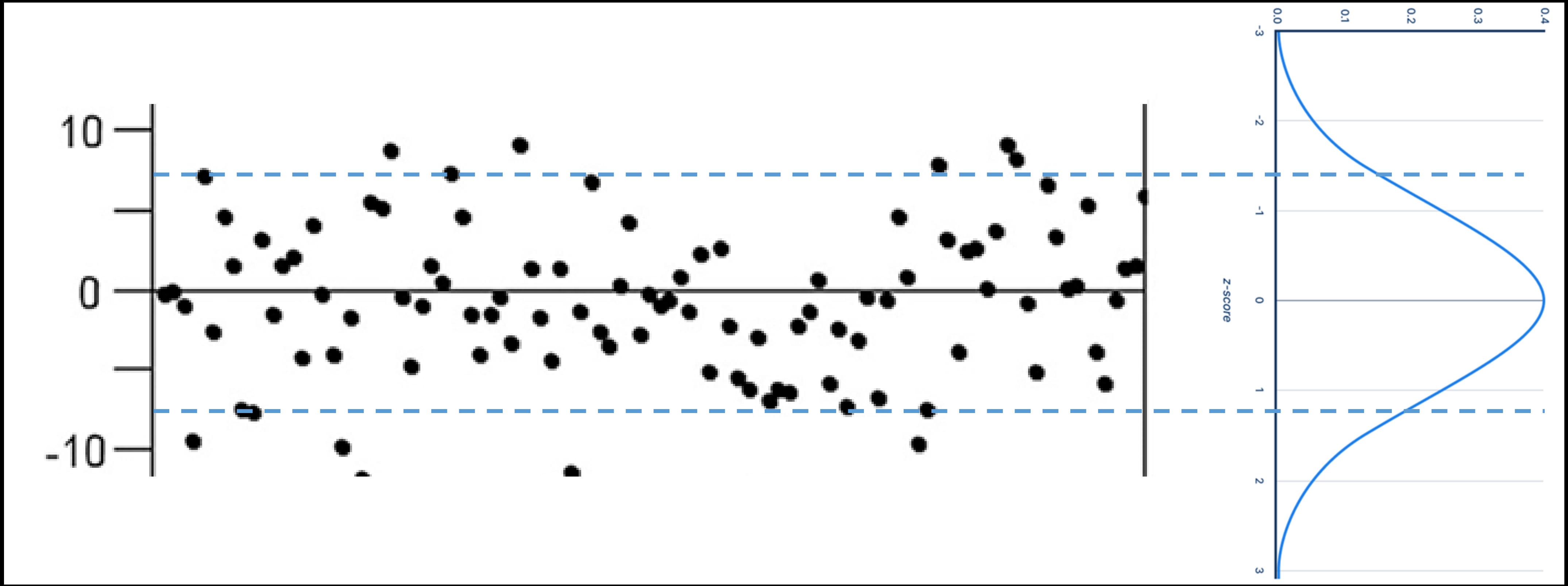
Implement a Cointegration Strategy

Cointegration Strategy

- Let’s create a cointegration strategy for banking sector stocks
  - 00005HK (i.e. HSBC)
  - 00011HK (i.e. Hang Seng Bank)
  - 00939HK (i.e. China Construction Bank)
  - 02388HK (i.e. Bank of China Hong Kong)
  - 03968HK (i.e. China Merchants Bank)

- Backtest Setup/ Assumptions
  - Instrument: 00005HK, 00011HK, 00939HK, 02388HK, 03968HK
  - Backtest period: 2021.01 – 2024.12
  - Initial capital: HK$100k
  - Short-selling are possible
  - We can trade odd lot

Market Entry/Exit based on the Spread's Distance



Cointegration Script (1)

```python
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.tsa.stattools import coint

class AlgoEvent:
    def __init__(self):
        self.order_size = 100
        self.entry_threshold = 1
        self.pair_id = 0
        self.contractSize = {}

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        for instrument in mEvt['subscribeList']:
            self.contractSize[instrument] = self.evt.getContractSpec(instrument)["contractSize"]
        self.evt.start()
```

Cointegration Script (2)

```
def on_bulkdatafeed(self, isSync, bd, ab):
    if not isSync:
        return
    # get historical data
    instruments = list(bd)
    data = {}
    for instrument in instruments:
        obs = self.evt.getHistoricalBar(contract={"instrument":instrument}, numOfBar=100, interval="D")
        data[instrument] = {t: obs[t]['c'] for t in obs}
    df = pd.DataFrame(data)
    df = df.fillna(method='ffill')
    df = df.dropna()
    self.evt.consoleLog("df=",df)

    # cointegration matrix and pair
    cointegrated_pairs, coint_df = find_cointegration_pair(instruments, df, threshold=0.05)
    self.evt.consoleLog("coint_df=",coint_df)

    self.evt.consoleLog("\nCointegrated Pairs:")
    for pair in cointegrated_pairs:
        self.evt.consoleLog(pair)
```

```
def find_cointegration_pair(instruments, df, threshold=0.05):
    n = len(instruments)
    coint_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(i + 1, n):
            p_value = coint(df[instruments[i]], df[instruments[j]])[1]
            coint_matrix[i, j] = p_value
    cointegrated_pairs = [(instruments[i], instruments[j]) for i in range(n) for j in
range(i + 1, n) if coint_matrix[i, j] < threshold]
    return cointegrated_pairs, coint_matrix
```

Cointegration Script (3)

```
# Find hedge ratios and trading signals
for stock1, stock2 in cointegrated_pairs:
    self.pair_id += 1
    # Fit linear regression to find hedge ratio
    Y = df[stock1]
    X = df[stock2]
    X = sm.add_constant(X)
    model = sm.OLS(Y, X).fit()
    hedge_ratio = model.params[1]

    # Calculate the spread and Z-score
    spread = Y - hedge_ratio * df[stock2]
    z_score = (spread - spread.mean()) / spread.std()
    self.evt.consoleLog('debug ... z_score=',z_score[-1])

    # short stock1 and long stock2
    if z_score[-1] > self.entry_threshold:
        order1 = AlgoAPIUtil.OrderObject(
            instrument = stock1,
            openclose = 'open',
            buysell = -1, #sell
            ordertype = 0, #market order
            volume = 1/self.contractSize[stock1]*self.order_size, #1 share of stock1
            orderRef = self.pair_id
        )
        self.evt.sendOrder(order1)
        order2 = AlgoAPIUtil.OrderObject(
            instrument = stock2,
            openclose = 'open',
            buysell = 1 if hedge_ratio>0 else -1,
            ordertype = 0, #market order
            volume = hedge_ratio/self.contractSize[stock2]*self.order_size,
            orderRef = self.pair_id
        )
        self.evt.sendOrder(order2)
```

```
    # long stock1 and short stock2
    elif z_score[-1] < -1*self.entry_threshold:
        order1 = AlgoAPIUtil.OrderObject(
            instrument = stock1,
            openclose = 'open',
            buysell = 1, #sell
            ordertype = 0, #market order
            volume = 1/self.contractSize[stock1]*self.order_size, #1 share of stock 1
            orderRef = self.pair_id
        )
        self.evt.sendOrder(order1)
        order2 = AlgoAPIUtil.OrderObject(
            instrument = stock2,
            openclose = 'open',
            buysell = -1 if hedge_ratio>0 else 1,
            ordertype = 0, #market order
            volume = hedge_ratio/self.contractSize[stock2]*self.order_size,
            orderRef = self.pair_id
        )
        self.evt.sendOrder(order2)
```

Cointegration Script (4)

```
# check order exit
pairs = {}
pos, osOrder, pendOrder = self.evt.getSystemOrders()
for tradeID in osOrder:
    order = osOrder[tradeID]
    instrument = order['instrument']
    pair_id = order['orderRef']
    buysell = order['buysell']
    volume = order['Volume']
    openprice = order['openprice']
    pl = order[‘pl']
    mktPrice = bd[instrument]['bidPrice'] if buysell==1 else bd[instrument]['askPrice']
    if pair_id not in pairs:
        pairs[pair_id] = {"entry_spread":0, "current_spread":0, "count":0, “pl":0}
    pairs[pair_id]["count"]+=1
    pairs[pair_id][“pl"]+= pl
    pairs[pair_id]["entry_spread"]+= -1*buysell*volume*openprice
    pairs[pair_id]["current_spread"]+= buysell*volume*mktPrice
for pair_id in pairs:
    if pairs[pair_id]["count"]==2 and ( \
        (pairs[pair_id]["entry_spread"]>0 and pairs[pair_id]["current_spread"]<=0) or \
        (pairs[pair_id]["entry_spread"]<0 and pairs[pair_id]["current_spread"]>=0) \
    ) and pairs[pair_id][“pl"]>0:
        order = AlgoAPIUtil.OrderObject(
            openclose = 'close',
            orderRef = pair_id
        )
        self.evt.sendOrder(order)
```

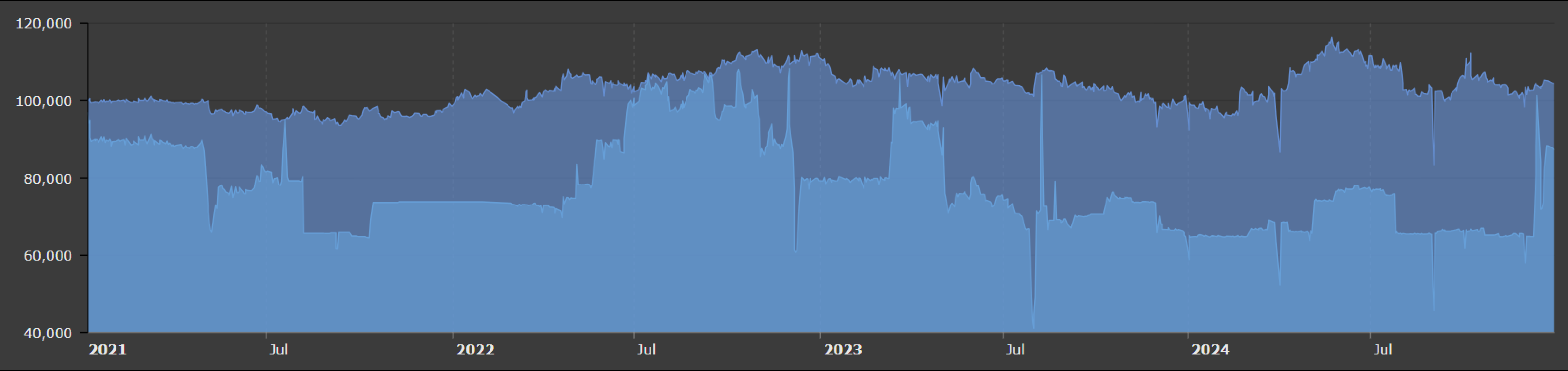
Full Script

```python
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.tsa.stattools
import coint

def find_cointegration_pair(instruments, df, threshold=0.05):
    n = len(instruments)
    coint_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(i + 1, n):
            p_value = coint(df[instruments[i]], df[instruments[j]])[1]
            coint_matrix[i, j] = p_value
    cointegrated_pairs = [(instruments[i], instruments[j]) for i in range(n) for j in range(i + 1, n) if
coint_matrix[i, j] < threshold]
    return cointegrated_pairs, coint_matrix

class AlgoEvent:
    def __init__(self):
        self.order_size = 100
        self.entry_threshold = 1
        self.pair_id = 0
        self.contractSize = {}

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        for instrument in mEvt['subscribeList']:
            self.contractSize[instrument] = self.evt.getContractSpec(instrument)["contractSize"]
        self.evt.start()
```

```python
def on_bulkdatafeed(self, isSync, bd, ab):
    if not isSync:
        return
    # get historical data
    instruments = list(bd)
    data = {}
    for instrument in instruments:
        obs = self.evt.getHistoricalBar(contract={"instrument":instrument}, numOfBar=100, interval="D")
        data[instrument] = {t: obs[t]['c'] for t in obs}
    df = pd.DataFrame(data)
    df = df.fillna(method='ffill')
    df = df.dropna()
    self.evt.consoleLog("df=",df)

    # cointegration matrix and pair
    cointegrated_pairs, coint_df = find_cointegration_pair(instruments, df, threshold=0.05)
    self.evt.consoleLog("coint_df=",coint_df)

    self.evt.consoleLog("\nCointegrated Pairs:")
    for pair in cointegrated_pairs:
        self.evt.consoleLog(pair)

    # Find hedge ratios and trading signals
    for stock1, stock2 in cointegrated_pairs:
        self.pair_id += 1
        # Fit linear regression to find hedge ratio
        Y = df[stock1]
        X = df[stock2]
        X = sm.add_constant(X)
        model = sm.OLS(Y, X).fit()
        hedge_ratio = model.params[1]

        # Calculate the spread and Z-score
        spread = Y - hedge_ratio * df[stock2]
        z_score = (spread - spread.mean()) / spread.std()
        self.evt.consoleLog('debug ... z_score=',z_score[-1])

        # short stock1 and long stock2
        if z_score[-1] > self.entry_threshold:
            order1 = AlgoAPIUtil.OrderObject(
                instrument = stock1,
                openclose = 'open',
                buysell = -1, #sell
                ordertype = 0, #market order
                volume = 1/self.contractSize[stock1]*self.order_size, #1 share of stock1
                orderRef = self.pair_id
            )
            self.evt.sendOrder(order1)
            order2 = AlgoAPIUtil.OrderObject(
                instrument = stock2,
                openclose = 'open',
                buysell = 1 if hedge_ratio>0 else -1,
                ordertype = 0, #market order
                volume = hedge_ratio/self.contractSize[stock2]*self.order_size,
                orderRef = self.pair_id
            )
            self.evt.sendOrder(order2)
        # long stock1 and short stock2
        elif z_score[-1] < -1*self.entry_threshold:
            order1 = AlgoAPIUtil.OrderObject(
                instrument = stock1,
                openclose = 'open',
                buysell = 1, #sell
                ordertype = 0, #market order
                volume = 1/self.contractSize[stock1]*self.order_size, #1 share of stock 1
                orderRef = self.pair_id
            )
            self.evt.sendOrder(order1)
            order2 = AlgoAPIUtil.OrderObject(
                instrument = stock2,
                openclose = 'open',
                buysell = -1 if hedge_ratio>0 else 1,
                ordertype = 0, #market order
                volume = hedge_ratio/self.contractSize[stock2]*self.order_size,
                orderRef = self.pair_id
            )
            self.evt.sendOrder(order2)
```

```python
# check order exit
pairs = {}
pos, osOrder, pendOrder = self.evt.getSystemOrders()
for tradeID in osOrder:
    order = osOrder[tradeID]
    instrument = order['instrument']
    pair_id = order['orderRef']
    buysell = order['buysell']
    volume = order['Volume']
    openprice = order['openprice']
    mktPrice = bd[instrument]['bidPrice'] if buysell==1 else bd[instrument]['askPrice']
    if pair_id not in pairs:
        pairs[pair_id] = {"entry_spread":0, "current_spread":0, "count":0, “pl”:0}
    pairs[pair_id]["count"]+=1
    pairs[pair_id][“pl”]+=pl
    pairs[pair_id]["entry_spread"]+= -1*buysell*volume*openprice
    pairs[pair_id]["current_spread"]+= buysell*volume*mktPrice
for pair_id in pairs:
    if pairs[pair_id]["count"]==2 and ( \
        (pairs[pair_id]["entry_spread"]>0 and pairs[pair_id]["current_spread"]<=0) or \
        (pairs[pair_id]["entry_spread"]<0 and pairs[pair_id]["current_spread"]>=0) \
    ) and pairs[pair_id][“pl”] >0:
        order = AlgoAPIUtil.OrderObject(
            openclose = 'close',
            orderRef = pair_id
        )
        self.evt.sendOrder(order)
```

Backtest Result



Year	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	YTD
2021	-0.13%	0.23%	-0.8%	0.12%	-1.98%	-0.48%	0.32%	-1.97%	-0.15%	0.84%	-0.0%	2.6%	-1.46%
2022	2.64%	1.76%	-1.91%	4.75%	-1.26%	-0.8%	1.99%	1.66%	1.71%	3.59%	-1.38%	-0.39%	12.79%
2023	-4.98%	0.46%	0.75%	-2.23%	2.29%	-1.01%	-4.01%	4.85%	-1.98%	-2.94%	-1.73%	1.69%	-8.91%
2024	-4.22%	4.87%	-5.37%	14.45%	1.91%	-0.62%	-3.75%	-15.98%	17.96%	-1.58%	-6.08%	5.99%	2.97%

Backtest Result

No. of Tradable Days:	983	No. of Win Days:	473	No. of Loss Days:	509
Win Rate:	48.167%	Max. Consecutive Win Day:	6	Max. Consecutive Loss Day:	9
Odd Ratio:	0.9293	Max. Consecutive Gains:	19026.24	Max. Consecutive Loss:	-20707.91
No. of Trades:	346	Average Consecutive Win Day:	0.82	Average Consecutive Loss Day:	0.97
Total PnL:	4223.64	Average Per Trade PnL:	12.21	Average Per Day PnL:	4.3
Mean Daily Return:	0.0159%	Median Daily Return:	-0.0167%	Mean Annual Return:	4.0162%
Daily Return StdDev:	1.5583%	25th pencentile Daily Return:	-0.4822%	75th pencentile Daily Return:	0.5076%
Daily Return Downside StdDev:	0.8775%	95% 1 day return VaR:	-1.5596%	99% 1 day return VaR:	-3.3032%
Daily Sharpe Ratio:	0.0102	Annual Sharpe Ratio:	0.1624	Max. Drawdown Amount:	32905.17
Daily Sortino Ratio:	0.0182	Annual Sortino Ratio:	0.2883	Max. Drawdown Percent:	28.3081%
Max. Drawdown Duration:	387	Average Drawdown Duration:	117.08	Annual Volatility:	24.6385%
Gross Profit:	75.7515	Gross Loss:	0.0	Profit Factor:	0.0
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	0.944
Calmar Ratio:	0.1419	Average Holding Day:	775.8193	Annual Turnover Rate:	196.5206%

Real Market Challenges

- **Model Risk**: Incorrect assumptions or overfitting can lead to poor performance.
- **Execution Risk**: Slippage and latency can affect the profitability of trades.
- **Market Risk**: Unforeseen market events can disrupt statistical relationships.
- **Regulatory Risk**: Changes in market regulations can impact strategy viability. (eg. short selling, leverage requirement, etc)

Wrap Up

- Statistical arbitrage is a popular strategy used by many quant trading firms that effectively make profits over years.
- From statistics point of view, it has a high probability of winning
- To run this strategy effectively,
  - Select the group of assets that are cointegrated
  - Try different observation period for cointegration analysis
  - Choose a distance measure to determine optimal entry and exit points
  - Data transformation of the original price series may increase significance of cointegration
  - May worth to rebalance portfolio due to hedge ratio changes

Overview of FX Market

Foreign Exchange Market

- The Foreign Exchange market, is also called FX or forex, is the largest asset class in terms of daily trading volume
- It has the following properties:
    1. Decentralized market
    2. Long Trading Hour
    3. High Turnover
    4. Low Spread
    5. Rollover Swap
    6. High Leverage

Centralized vs Decentralized Market

- FX market operates by the market itself. There is no single entity responsible for central clearing.
- More price transparency and less market manipulation

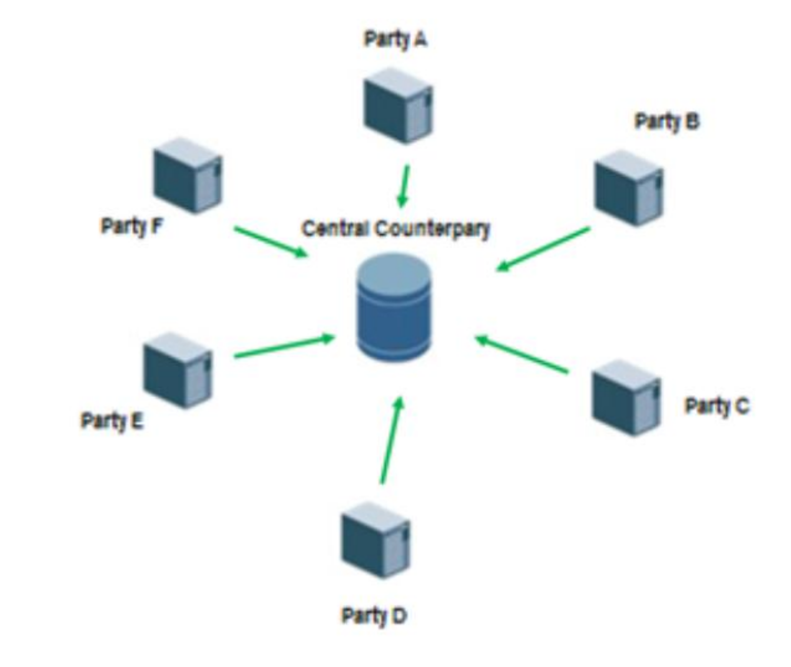


Figure 1 - Central Clearing For Equity Market

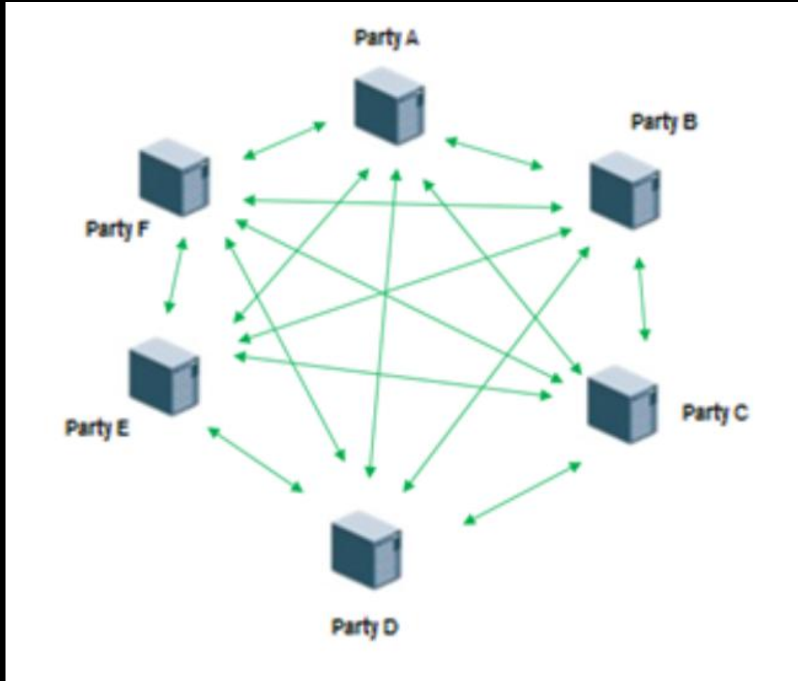
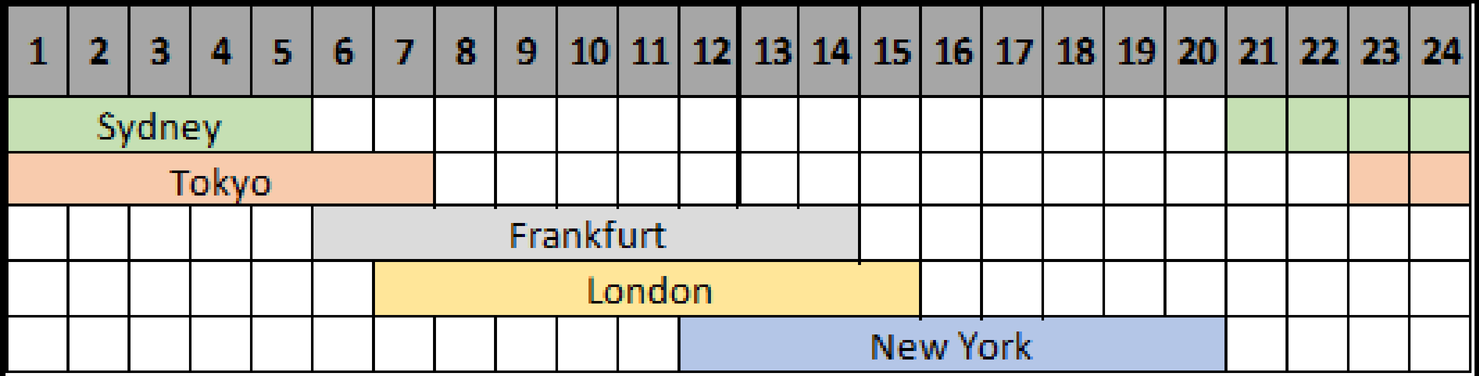


Figure 2 - Bilateral Clearing Nature in FX Market

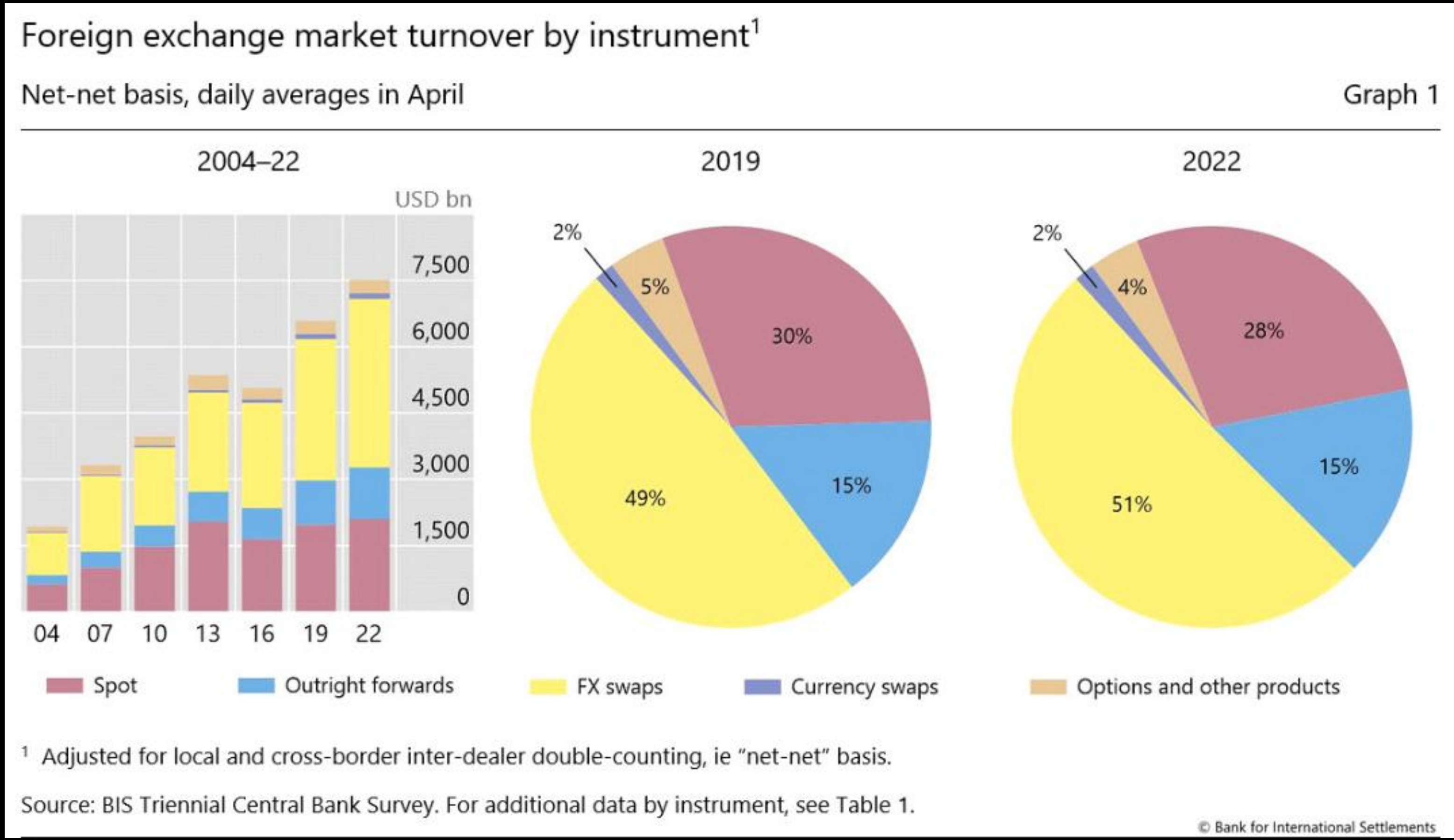
Long Trading Hour

- FX market operates in 24*5.5
- It starts trading from 21:00 GMT in Sydney on Sunday to 20:00 GMT in New York on Friday
  - i.e. Monday 5am (HKT) to Saturday 4am (HKT)
- More trading opportunities
- Less market gap and price jumps



High Turnover

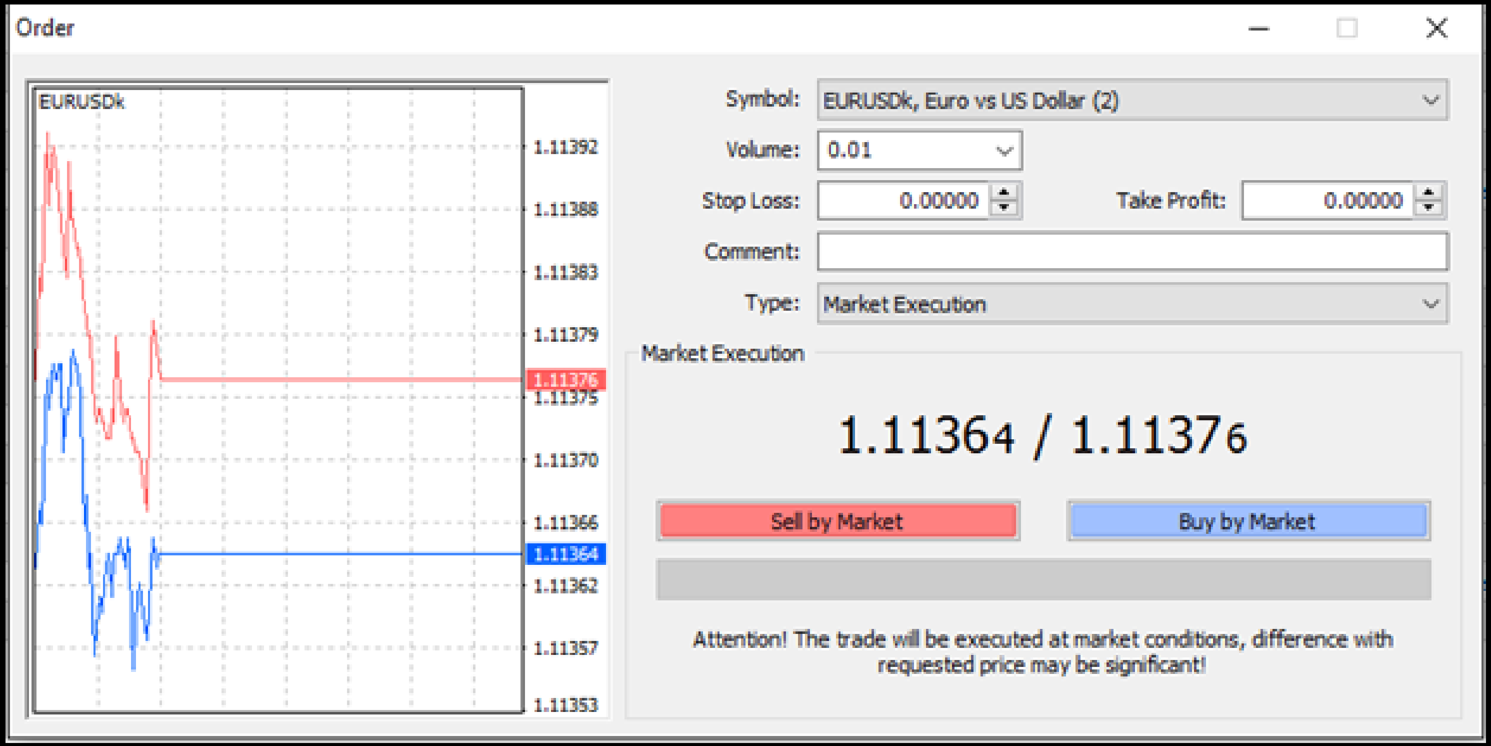
- Global equity market: $3.93 trillion
- Forex: $6.6 trillion



Low Spread

- For EURUSD, the percentage spread is

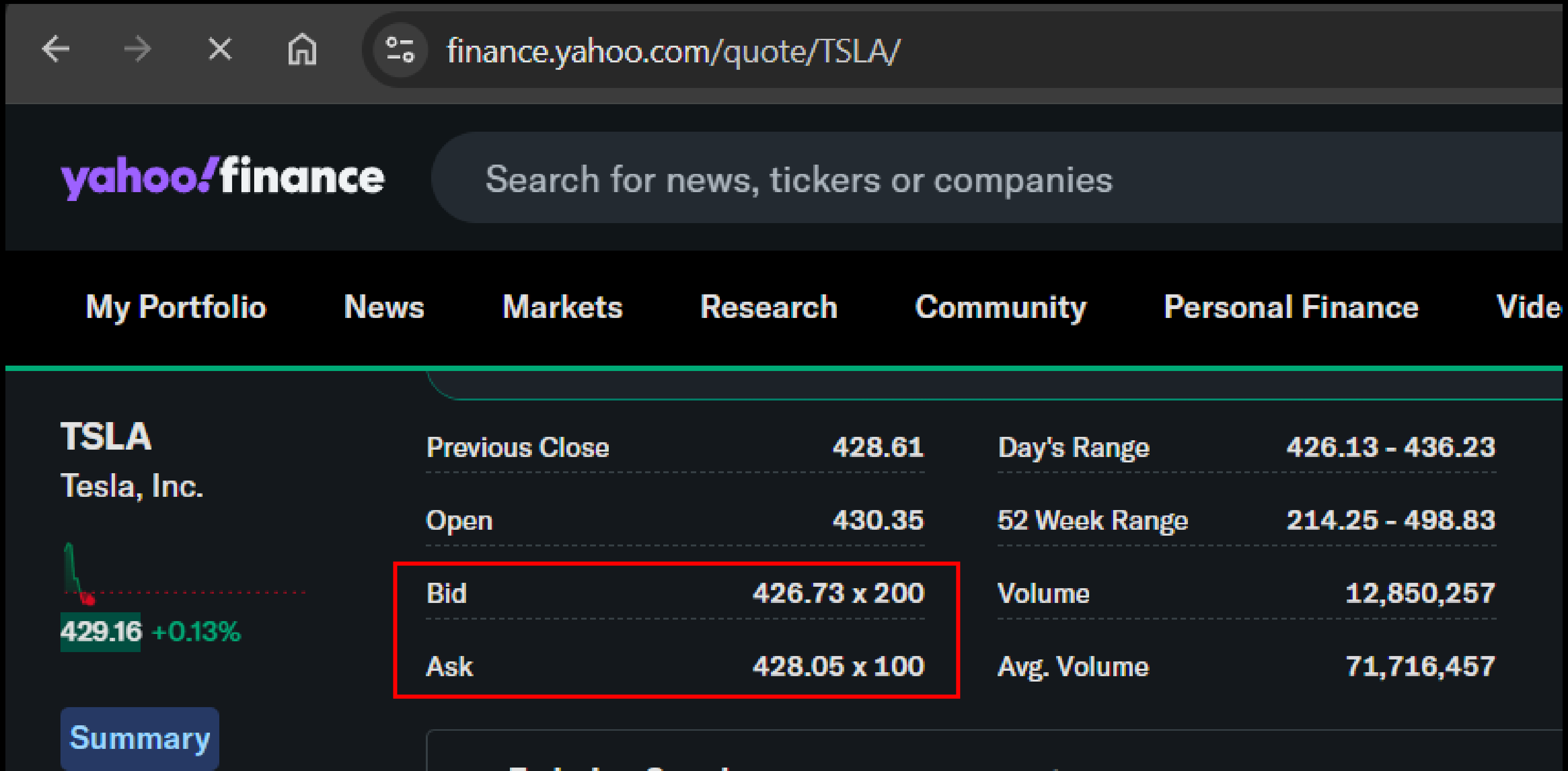
$$(1.11376 - 1.11364)/((1.11376+1.11364)/2\ ) = 0.0108\%$$



Low Spread

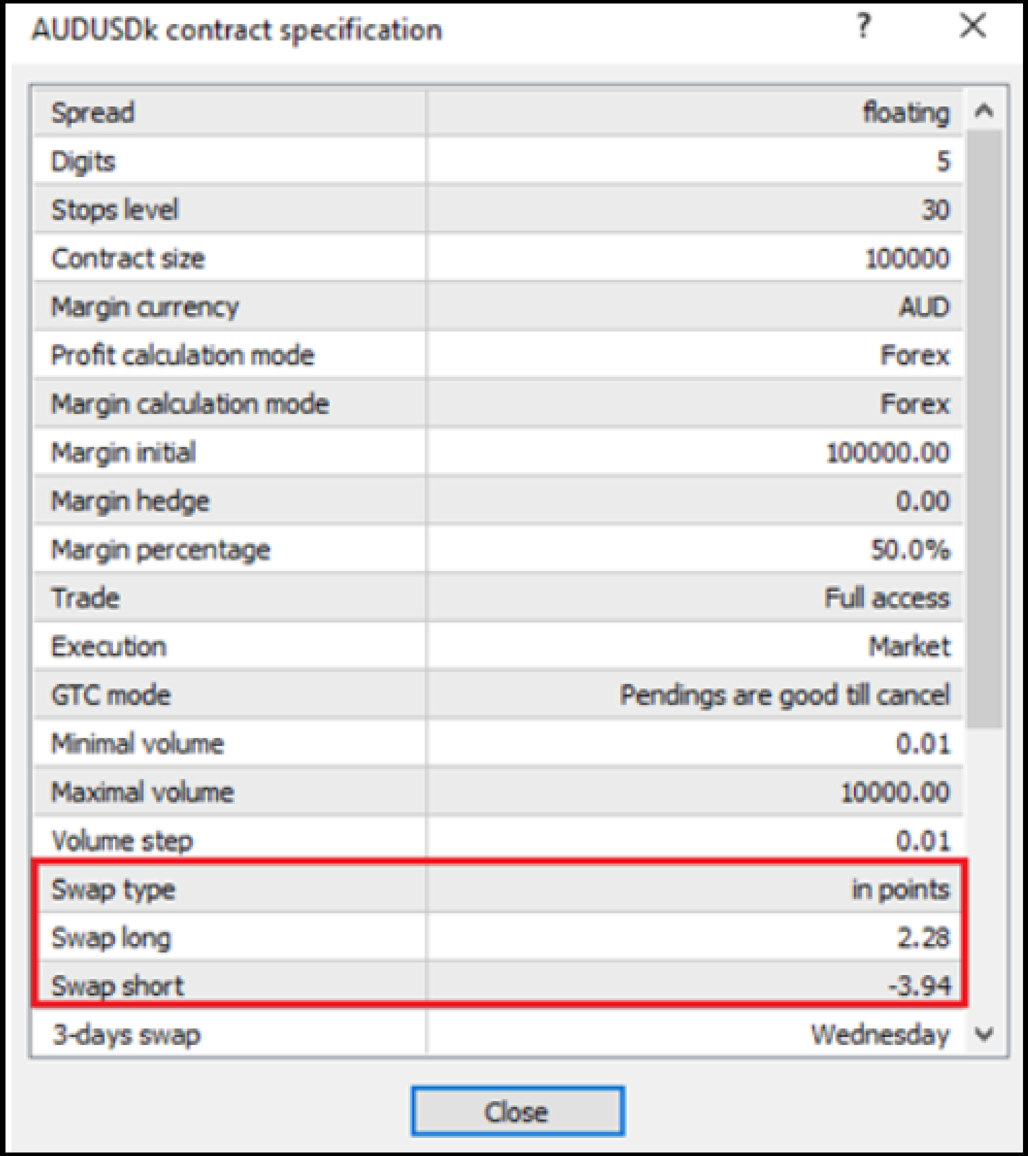
- For TSLA, the percentage spread is

  (428.05 – 426.73)/((428.05 + 426.73)/2) = 0.031%



Rollover Swap

- FX trading involves buying 1 currency and selling another currency
- You will receive interest from the currency you buy; and pay interest for the currency you sell
- If the net interest is positive, you will have positive carry for holding your position overnight.
- The net interest rate is called “overnight swap”, or simply “swap”
- The swap rates are updating every day based on the funding costs the FX brokers need to maintain its risks and operations
- For the same instrument, the swap rates can be different for different FX brokers



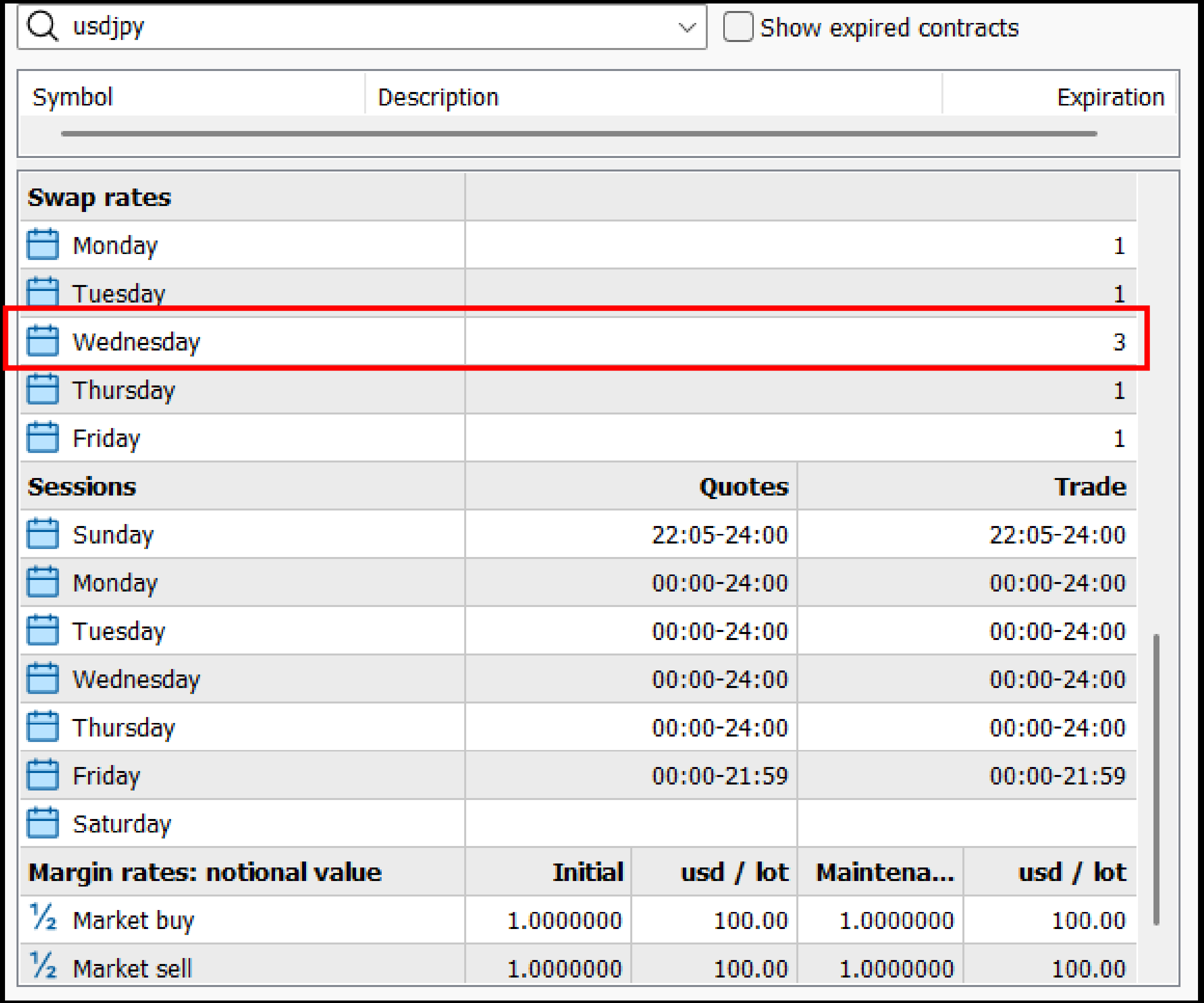
AUDUSDk contract specification

Spread	floating
Digits	5
Stops level	30
Contract size	100000
Margin currency	AUD
Profit calculation mode	Forex
Margin calculation mode	Forex
Margin initial	100000.00
Margin hedge	0.00
Margin percentage	50.0%
Trade	Full access
Execution	Market
GTC mode	Pendings are good till cancel
Minimal volume	0.01
Maximal volume	10000.00
Volume step	0.01
Swap type	in points
Swap long	2.28
Swap short	-3.94
3-days swap	Wednesday

Close

Swap Schedule

- The swap payment, if any, will be settled in your broker account every day.
- To handle market closure in weekends, the swap fee will be calculated 3 times on Wednesday
- That means if you have overnight position on Wednesday, you will receive/pay 3 times of the swap fee.



High Leverage

- Compared to stock market, FX is less regulated due to the decentralization property
- Many FX brokers can allow users to trade at 100x leverage. Some even supports 5000x
- For a Hong Kong regulated FX broker, investors can only trade up to 5x leverage

Market Quotation

- In FX market, it is usually quoted as **XXXYYY**
- It means that for 1 unit of XXX, how much YYY can you get?

Market Watch: 21:58:56

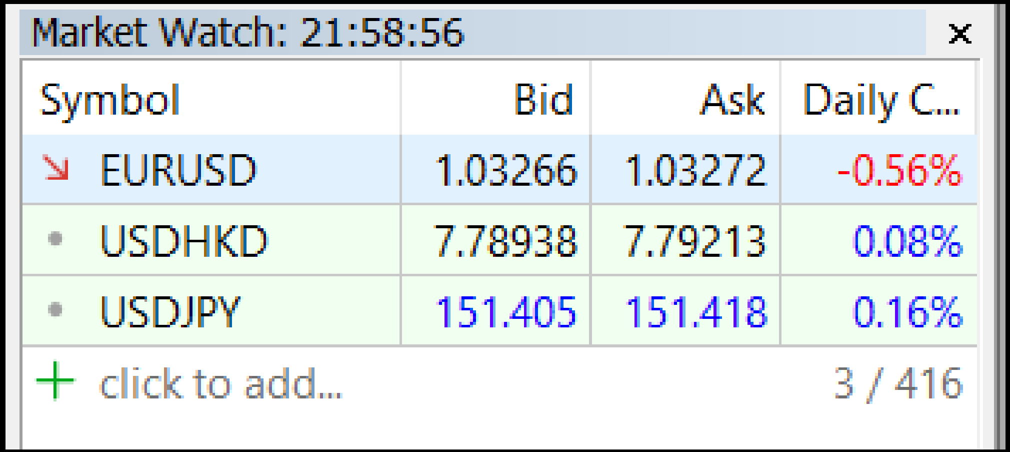
Symbol	Bid	Ask	Daily C...
EURUSD	1.03266	1.03272	-0.56%
USDHKD	7.78938	7.79213	0.08%
USDJPY	151.405	151.418	0.16%
+ click to add...			3 / 416

Market Quotation

- Some brokers may quote in the form of **YYY/XXX**
- It means that for 1 unit of XXX, how much YYY can you get?

Which currency do I buy?

- For stock, a price quotation is $xxx / 1 share of yyy
- For currency, a price is quoted in the amount of currency XXX / 1 unit of currency YYY.
  - 7.79 simple means 7.79 HKD / 1 USD



Market Watch: 21:58:56

Symbol	Bid	Ask	Daily C...
EURUSD	1.03266	1.03272	-0.56%
USDHKD	7.78938	7.79213	0.08%
USDJPY	151.405	151.418	0.16%
click to add...			3 / 416

Mostly Traded Currencies

Most traded currencies by value

Currency distribution of global foreign exchange market turnover[1]

V · T · E

	Currency	ISO 4217 code	Symbol or Abbrev.[2]	Proportion of daily volume		Change (2019–2022)
				April 2019	April 2022	
1	U.S. dollar	USD	$, US$	88.3%	88.5%	▲ 0.2pp
2	Euro	EUR	€	32.3%	30.5%	▼ 1.8pp
3	Japanese yen	JPY	¥, 円	16.8%	16.7%	▼ 0.1pp
4	Sterling	GBP	£	12.8%	12.9%	▲ 0.1pp
5	Renminbi	CNY	¥, 元	4.3%	7.0%	▲ 2.7pp
6	Australian dollar	AUD	$, A$	6.8%	6.4%	▼ 0.4pp
7	Canadian dollar	CAD	$, Can$	5.0%	6.2%	▲ 1.2pp
8	Swiss franc	CHF	Fr., fr.	4.9%	5.2%	▲ 0.3pp
9	Hong Kong dollar	HKD	$, HK$, 元	3.5%	2.6%	▼ 0.9pp
10	Singapore dollar	SGD	$, S$	1.8%	2.4%	▲ 0.6pp
11	Swedish krona	SEK	kr, Skr	2.0%	2.2%	▲ 0.2pp
12	South Korean won	KRW	₩, 원	2.0%	1.9%	▼ 0.1pp
13	Norwegian krone	NOK	kr, Nkr	1.8%	1.7%	▼ 0.1pp
14	New Zealand dollar	NZD	$, $NZ	2.1%	1.7%	▼ 0.4pp
15	Indian rupee	INR	₹	1.7%	1.6%	▼ 0.1pp
16	Mexican peso	MXN	$, Mex$	1.7%	1.5%	▼ 0.2pp
17	New Taiwan dollar	TWD	$, NT$, 圓	0.9%	1.1%	▲ 0.2pp
18	South African rand	ZAR	R	1.1%	1.0%	▼ 0.1pp

Sum of column = 200%

Reference: https://en.wikipedia.org/wiki/Template:Most_traded_currencies

Trading USDHKD Peg

[redacted]
Sponsored
USDHKD, buy 0.01 2022.12.02 10:53:37
7.78153 → 7.84256 7.78
USDHKD, buy 1.00 2022.12.05 08:23:24
7.76894 → 7.81293 563.04
USDHKD, buy 1.00 2022.12.06 12:43:21
7.77864 → 7.84254 814.79
USDHKD, buy 1.00 2022.12.07 10:45:00
7.79459 → 7.84256 611.66
USDHKD, buy 1.00 2022.12.12 15:30:57
7.77458 → 7.84256 866.81
USDHKD, buy 1.00 2022.12.19 15:09:58
7.77786 → 7.84255 824.86
USDHKD, buy 2.00
7.79600 → 7.80
258.77
USDHKD, buy 1.00
7.80770 → 7.84256
444.50
USDHKD,
7.85000
24.20
必賺！
USDHKD買賣方法
Profit: 5 865.39
Credit:
USD利潤
Deposit: 0.00
Withdrawal: 0.00
Balance: 5 865.39
聯繫匯率保底必定賺錢！
LEARN MORE

Sure Win???

What is a Currency Peg?

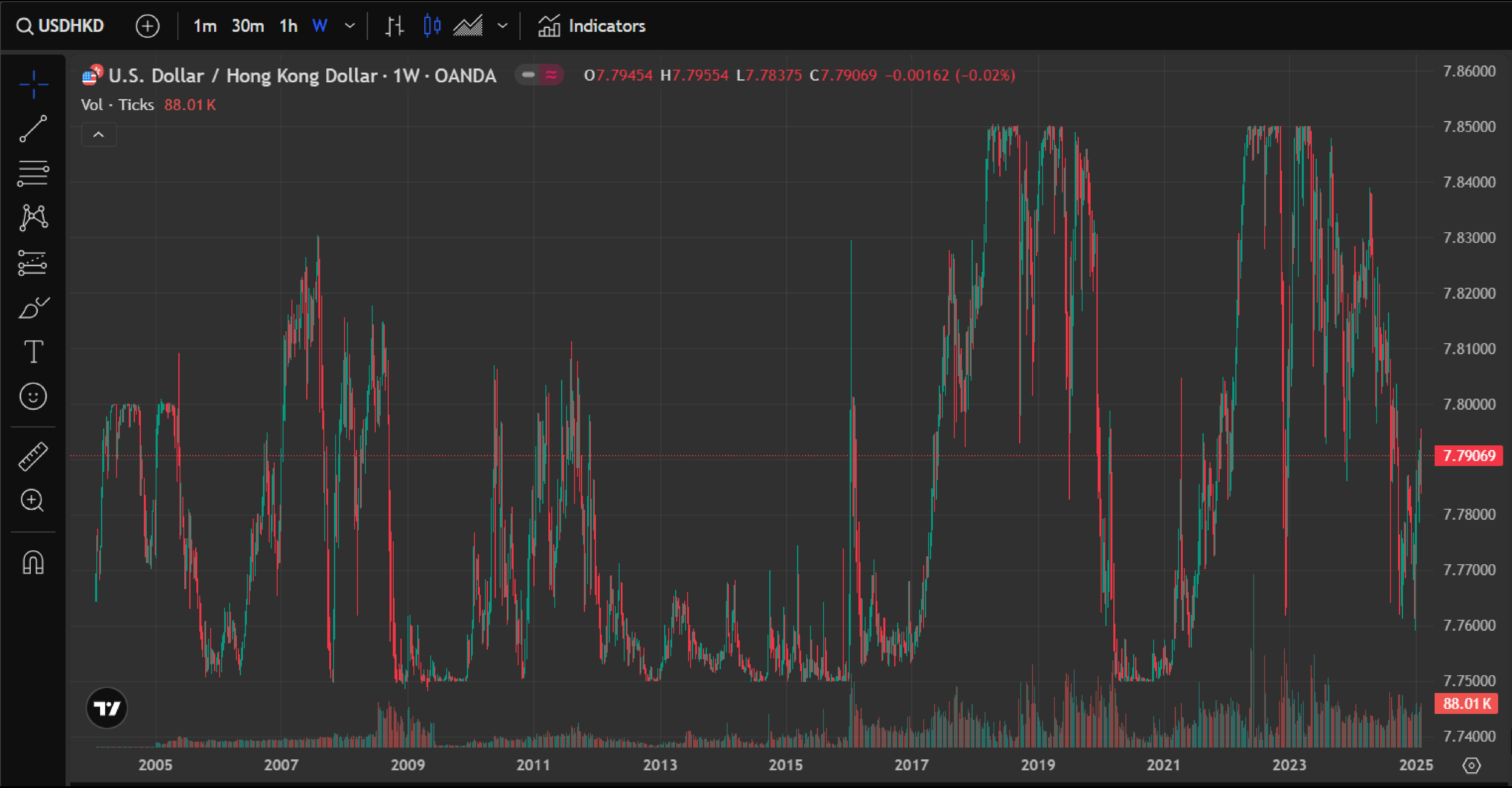
- **Definition**: A currency peg is a policy where a country's currency value is tied or fixed to another major currency.
- **Purpose**: To stabilize the exchange rate and provide predictability for international trade.

The USDHKD Peg

- In 1983, the Hong Kong Gov introduced to peg the exchange rate with USD.
- The Hong Kong Dollar (HKD) is pegged to the US Dollar (USD) at approximately 7.8 HKD to 1 USD.
- The Hong Kong Monetary Authority (HKMA) maintains this peg through market interventions.

Pros & Cons of USDHKD peg

Pros	Cons
Stability: Reduces exchange rate volatility for businesses and investors.	**Loss of Monetary Policy Autonomy**: HKMA cannot freely adjust interest rates.
Trade Facilitation: Simplifies international trade with the US	**Vulnerability to External Shocks**: Economic issues in the US can impact Hong Kong.
Inflation Control: Helps maintain low inflation rates in Hong Kong	**Speculative Attacks**: Potential for market speculation against the peg.


USDHKD
1m
30m
1h
W
Indicators
U.S. Dollar / Hong Kong Dollar · 1W · OANDA
O7.79454 H7.79554 L7.78375 C7.79069 −0.00162 (−0.02%)
Vol · Ticks 88.01 K
7.86000
7.85000
7.84000
7.83000
7.82000
7.81000
7.80000
7.79069
7.78000
7.77000
7.76000
7.75000
88.01 K
7.74000
2005
2007
2009
2011
2013
2015
2017
2019
2021
2023
2025

The USDHKD Peg

- Allows fluctuation between 7.75 and 7.85
- If the exchange rate is approaching 7.75, that means HKD is getting strong (or USD is weak). HKMA will sell HKD and buy USD in the market
- If the exchange rate is approaching 7.85, that means HKD is getting weak (or USD is strong). HKMA will buy HKD and sell USD in the market

How retail investors can capture the opportunities?

- You want to buy around 7.75 and sell around 7.85 to capture the 0.1 profits
- Suppose you have 1m HKD at the beginning,
  - At 7.75, you should sell all HKD to get 1m/7.75 = 129,032 USD
  - At 7.85, you then sell all USD to get 129,032*7.85 = 1,012,903 HKD

Is the USDHKD peg strategy safe?

- Can international hedge funds still attack the peg mechanism just like Asia Financial Crisis in 1997?
- What if HKMA run out of capital to interfere the market?

George Soros

Is the USDHKD peg strategy safe?

- If HKD is depreciating toward 7.85, that means the market is dumping HKD.
- HKMA has sufficient reserve to buy back all circulating HKD in the market.

Press Releases

06 Feb 2026

Hong Kong's Latest Foreign Currency Reserve Assets Figures Released

The Hong Kong Monetary Authority (HKMA) announced today (6 February) that the official foreign currency reserve assets of Hong Kong amounted to US$435.6 billion as at the end of January 2026 (end-December 2025: US$427.9 billion) (Annex).

There were no unsettled foreign exchange contracts at end-January 2026 and end-December 2025.

The total foreign currency reserve assets of US$435.6 billion represent over five times the currency in circulation or about 38% of Hong Kong dollar M3.

Reference: https://www.hkma.gov.hk/eng/news-and-media/press-releases/2026/02/20260206-3/

Is the USDHKD peg strategy safe?

- If the exchange rate is moving toward 7.75 (i.e. HKD is appreciating), that means the market has growing demand for HKD.
- HKMA can easily print more HKD

Simple USD/HKD trading strategy

Trading Logic:

- Get the current market price of USD/HKD
- Entry logic:
  - If current position is zero,
    - If market price is less than 7.76, open a buy order with all available capital
    - If market price is great than 7.84, open a sell order with all available capital
- Exit logic:
  - If current position is non-zero,
    - If it previously buy at 7.76 but now rebound to 7.8 or above, close order
    - If it previously sell at 7.84 but now reverse to 7.8 or below, close order

Backtest

- Settings
    - Instrument: USDHKD
    - Period: 2020-01 to 2024-12
    - Enable short-sell: YES
    - Leverage: 1x
    - Data Interval: 1-day
    - Transaction Cost: 0
    - Initial Capital: 100k HKD

Backtest

- Define variables at initialization

```
class AlgoEvent:
    def __init__(self):
        self.thre_b = 7.76
        self.thre_s = 7.84
        self.thre_exit = 7.8
        self.timer = datetime(1970,1,1)
```

Backtest

- Get contract size for USDHKD
- In forex market, 1 lot = 100,000 unit of settle currency (i.e. HKD)

```
def start(self, mEvt):
    self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)

    # get contract size of USDHKD
    instrument = mEvt['subscribeList'][0]
    spec = self.evt.getContractSpec(instrument)
    self.contractSize = spec["contractSize"]

    self.evt.start()
```

Backtest

- Get current position

```
def on_marketdatafeed(self, md, ab):
    if md.timestamp>=self.timer+timedelta(hours=24):
        self.timer = md.timestamp

        # get current position
        pos, osOrder, pendOrder = self.evt.getSystemOrders()
        position = pos[md.instrument]["netVolume"]
```

Backtest

- Order Entry

```
if position==0:
    # calculate the max lot can trade based on current balance
    lotSize = ab['availableBalance']/self.contractSize/md.askPrice
    lotSize = round( 0.01*floor(100*lotSize), 2)

    if md.askPrice<=self.thre_b:
        order = AlgoAPIUtil.OrderObject(
            instrument = md.instrument,
            buysell = 1,
            ordertype = 0, #market order
            openclose = 'open’,
            volume = lotSize,
            orderRef = 'Long’
        )
        self.evt.sendOrder(order)
    elif md.bidPrice>=self.thre_s:
        order = AlgoAPIUtil.OrderObject(
            instrument = md.instrument,
            buysell = -1,
            ordertype = 0, #market order
            openclose = 'open’,
            volume = lotSize,
            orderRef = 'Short’
        )
        self.evt.sendOrder(order)
```

Backtest

- Order Exit

```
# order exit
elif position>0 and md.bidPrice>=self.thre_exit:
    order = AlgoAPIUtil.OrderObject(
        openclose = 'close',
        orderRef = 'Long'
    )
    self.evt.sendOrder(order)
elif position<0 and md.askPrice<=self.thre_exit:
     order = AlgoAPIUtil.OrderObject(
         openclose = 'close',
         orderRef = 'Short'
     )
     self.evt.sendOrder(order)
```

Backtest

- Full script

```python
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
from datetime import datetime, timedelta
from math import floor

class AlgoEvent:
    def __init__(self):
        self.thre_b = 7.76
        self.thre_s = 7.84
        self.thre_exit = 7.8
        self.timer = datetime(1970,1,1)

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)

        # get contract size of USDHKD
        instrument = mEvt['subscribeList'][0]
        spec = self.evt.getContractSpec(instrument)
        self.contractSize = spec["contractSize"]

        self.evt.start()
```

```python
def on_marketdatafeed(self, md, ab):
    if md.timestamp>=self.timer+timedelta(hours=24):
        self.timer = md.timestamp

        # get current position
        pos, osOrder, pendOrder = self.evt.getSystemOrders()
        position = pos[md.instrument]["netVolume"] # order entry

        if position==0:
            # calculate the max lot can trade based on current balance
            lotSize = ab['availableBalance']/self.contractSize/md.askPrice
            lotSize = round( 0.01*floor(100*lotSize), 2)

            if md.askPrice<=self.thre_b:
                order = AlgoAPIUtil.OrderObject(
                    instrument = md.instrument,
                    buysell = 1,
                    ordertype = 0, #market order
                    openclose = 'open',
                    volume = lotSize,
                    orderRef = 'Long'
                )
                self.evt.sendOrder(order)
            elif md.bidPrice>=self.thre_s:
                order = AlgoAPIUtil.OrderObject(
                    instrument = md.instrument,
                    buysell = -1,
                    ordertype = 0, #market order
                    openclose = 'open',
                    volume = lotSize,
                    orderRef = 'Short'
                )
                self.evt.sendOrder(order)

        # order exit
        elif position>0 and md.bidPrice>=self.thre_exit:
            order = AlgoAPIUtil.OrderObject(
                openclose = 'close',
                orderRef = 'Long'
            )
            self.evt.sendOrder(order)
        elif position<0 and md.askPrice<=self.thre_exit:
            order = AlgoAPIUtil.OrderObject(
                openclose = 'close',
                orderRef = 'Short'
            )
            self.evt.sendOrder(order)
```

Backtest Results

Return is too small

Monthly Return:

Year	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	YTD
2020	0.0%	0.0%	-0.08%	-0.01%	-0.0%	-0.01%	-0.0%	-0.0%	0.0%	0.03%	-0.01%	-0.0%	-0.08%
2021	0.01%	0.04%	0.18%	-0.06%	-0.08%	0.08%	0.07%	0.05%	0.08%	-0.03%	0.2%	0.05%	0.6%
2022	0.0%	0.0%	0.0%	-0.06%	-0.0%	-0.02%	-0.01%	0.01%	-0.05%	0.04%	0.53%	0.3%	0.74%
2023	0.0%	-0.06%	-0.02%	0.02%	0.22%	-0.05%	0.43%	-0.03%	0.13%	0.08%	0.31%	0.0%	1.02%
2024	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%

No. of Tradable Days:	1547	No. of Win Days:	475	No. of Loss Days:	457
Win Rate:	50.9657%	Max. Consecutive Win Day:	8	Max. Consecutive Loss Day:	8
Odd Ratio:	1.0394	Max. Consecutive Gains:	412.56	Max. Consecutive Loss:	-241.92
No. of Trades:	8	Average Consecutive Win Day:	1.33	Average Consecutive Loss Day:	0.54
Total PnL:	2297.28	Average Per Trade PnL:	287.16	Average Per Day PnL:	1.48
Mean Daily Return:	0.0015%	Median Daily Return:	0.0%	Mean Annual Return:	0.3876%
Daily Return StdDev:	0.0263%	25th pencentile Daily Return:	-0.0017%	75th pencentile Daily Return:	0.0025%
Daily Return Downside StdDev:	0.0152%	95% 1 day return VaR:	-0.0318%	99% 1 day return VaR:	-0.0788%
Daily Sharpe Ratio:	0.0585	Annual Sharpe Ratio:	0.929	Max. Drawdown Amount:	344.28
Daily Sortino Ratio:	0.1013	Annual Sortino Ratio:	1.6078	Max. Drawdown Percent:	0.3432%
Max. Drawdown Duration:	287	Average Drawdown Duration:	42.42	Annual Volatility:	0.4156%
Gross Profit:	2297.28	Gross Loss:	0.0	Profit Factor:	0.0
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treunor Ratio:	0.0	Tail Ratio:	1.1842
Calmar Ratio:	1.1294	Average Holding Day:	279.75	Annual Turnover Rate:	60.3091%

Some ways to improve the strategy

- Earlier entry so that it can trade more
- Exit earlier such that the holding period can be shorter
- Split the capital and trade at a different price level, eg.
  - 0.1 lot at 7.77
  - 0.2 lot at 7.76
  - 0.3 lot at 7.75
- Engage in leverage trading

Is it executable in the market?

- Max profit = 7.85 / 7.75 – 1 = 1.29%
- The percentage spread = 2*(7.8246 – 7.7568)/(7.8246 + 7.7568) = 0.87%

Personal | Business | Global Banking and Markets | Global Private Banking   English   Log on

HSBC   Banking Accounts & Services | Borrowing Cards & Loans | Investing Securities & Currency Exchange | Insurance Protection & Planning | Insights Analysis & Market Data | Offers Latest Rewards

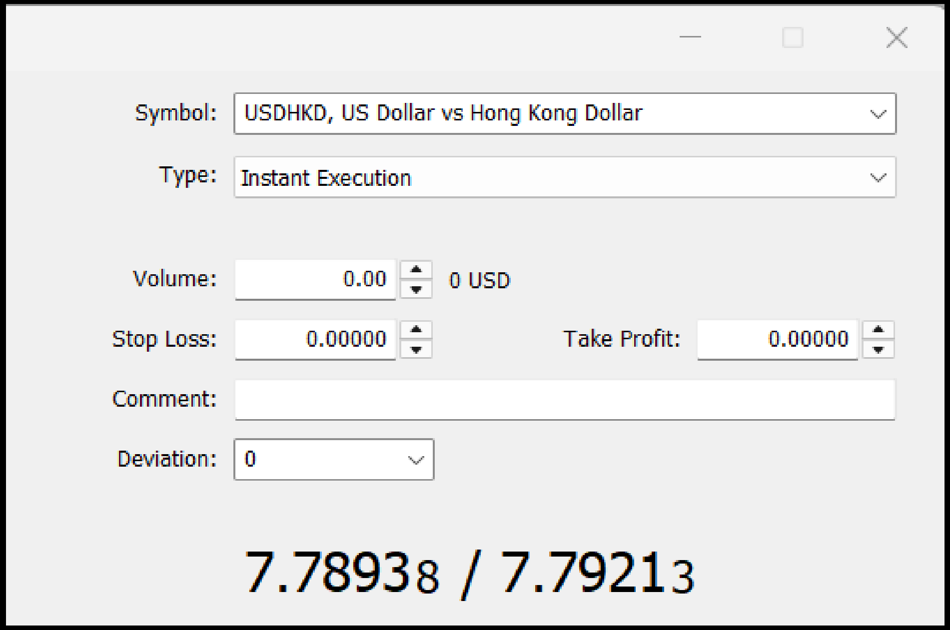
Exchange rates of foreign currencies against HK Dollar

Currency	Telegraphic Transfer Bank Buy	Telegraphic Transfer Bank Sell	Banknotes Bank Buy	Banknotes Bank Sell	Last Updated
US Dollar	7.75680	7.82460	7.72480	7.85670	as at 9 Feb 2025 00:00 HKT

Reference: https://www.hsbc.com.hk/investments/products/foreign-exchange/currency-rate/

How about other CFD brokers?

- The percentage spread = 2*(7.79213 - 7.78938)/(7.8938 + 7.79213) = 0.035%



High Swap Fee

- For long position, we are paying 9.3/100000 = 0.0093% every day
  - So 1.29% / 0.0093% = 139 days
- For short position, we are paying 0.0194% every day
  - So 1.29% / 0.0194% = 66 days

USDHKD, USD/HKD, US Dollar vs Hong Kong Dollar		
Category	Exotic	
Digits	5	
Contract size	100 000 USD	
Spread	floating	
Stops level	0	
Margin currency	USD	
Profit currency	HKD	
Calculation	Forex	
Chart mode	By bid price	
Trade	Full access	
Execution	Instant	
GTC mode	Good till cancelled	
Filling	Fill or Kill	
Expiration	All	
Orders	All	
Minimal volume	0.01	
Maximal volume	200	
Volume step	0.01	
Swap type	In points	
Swap long	-9.3	
Swap short	-19.4	
Swap rates		
Monday	1	
Tuesday	1	
Wednesday	3	
Thursday	1	
Friday	1	
Sessions	**Quotes**	**Trade**
Sunday	22:05-24:00	22:05-24:00

Wrap Up

- Without leverage, the maximum profit you can get is 7.85/7.75 – 1 = 1.29%
- It usually takes a few months for the price to rebound.

- For Spot market,
  - Due to physical cash exchange, no leverage involved.
  - The bid-ask spread are also too wide for currency exchanges.

  → Not profitable

- For CFD market,
  - We can take leverage
  - Acceptable bid-ask spread
  - However overnight swap fee is too high

  → Not as profitable as expected

Final Conclusion

- This strategy has a high winning rate.
- But it requires a good market timing for entry and exit.
- Should keep your holding time short, preferably within a few days.
- No free lunch in the world!!!

Key Takeaways

- Statistical arbitrage is a type of strategy commonly used by many hedge funds and investment banks.
- Co-integration is used to identify the basket of suitable instruments that can construct a stationary time series
- Correlation measures short terms relation while cointegration for long terms
- FX Market has its unique properties and features
- Discuss the risks and opportunities of the USD-HKD arbitrage